

Week 6

PINNs

Nebius Academy



Fine tuning

Transfer learning: pre-train simple, then fine tune

A pre-trained model

Fine
tuning

Consumer model

Trained

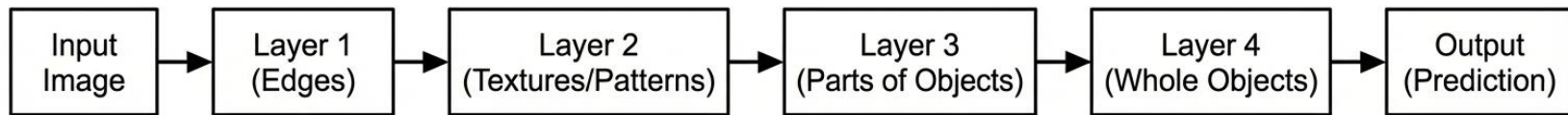
- For a simple task
- On abundant data
- By a rich corporation or a cool research lab

Fine tuned

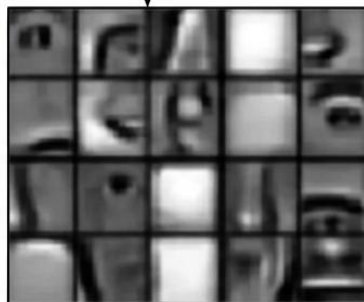
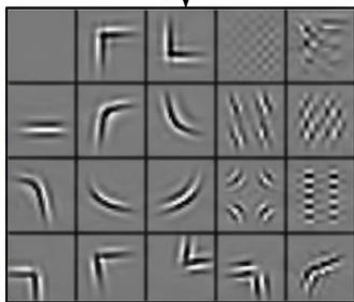
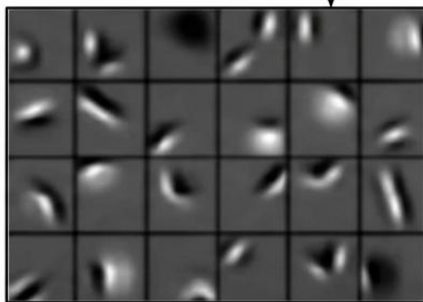
- For a more niche task
- On scarce data

Features inside the neural network

Neural Network
Computational
Graph



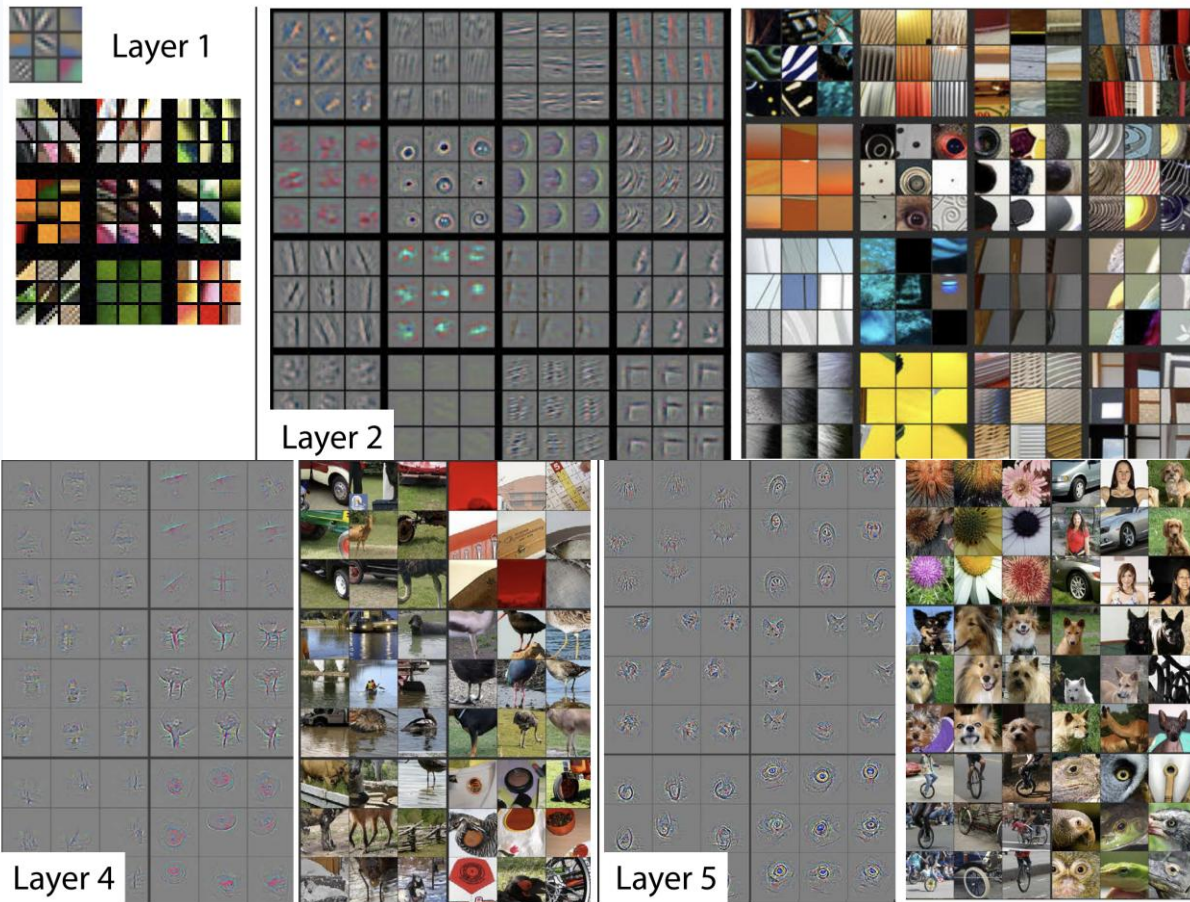
Feature
Progression



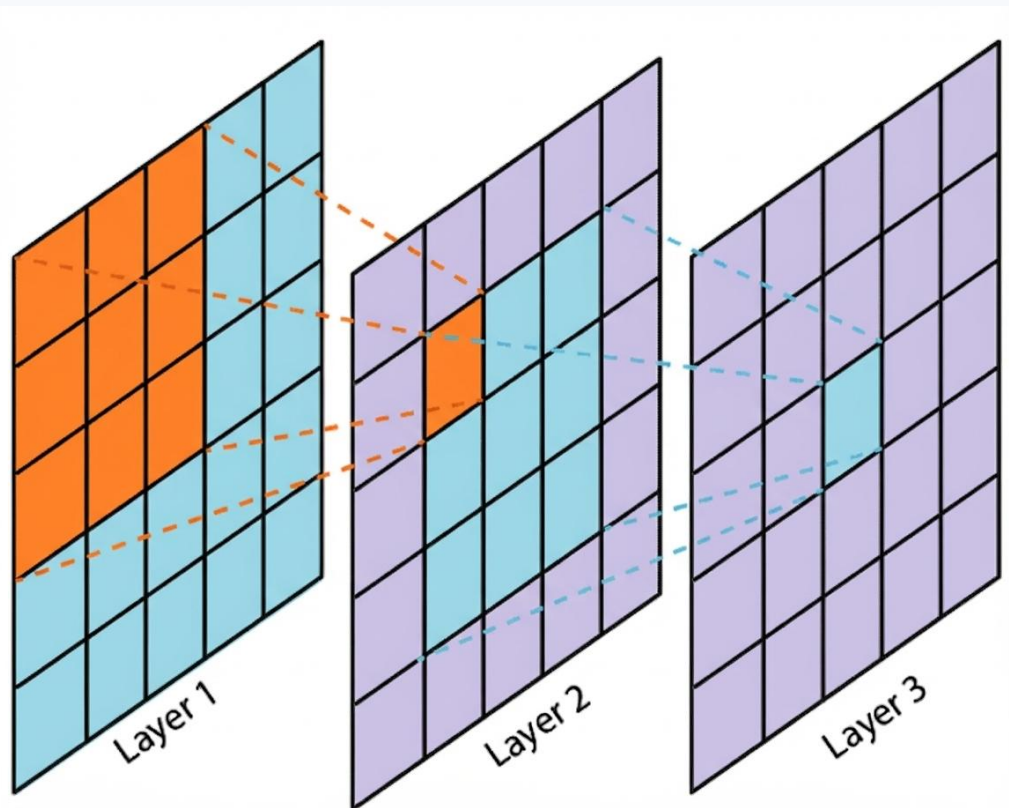
lower-level features

higher-level features

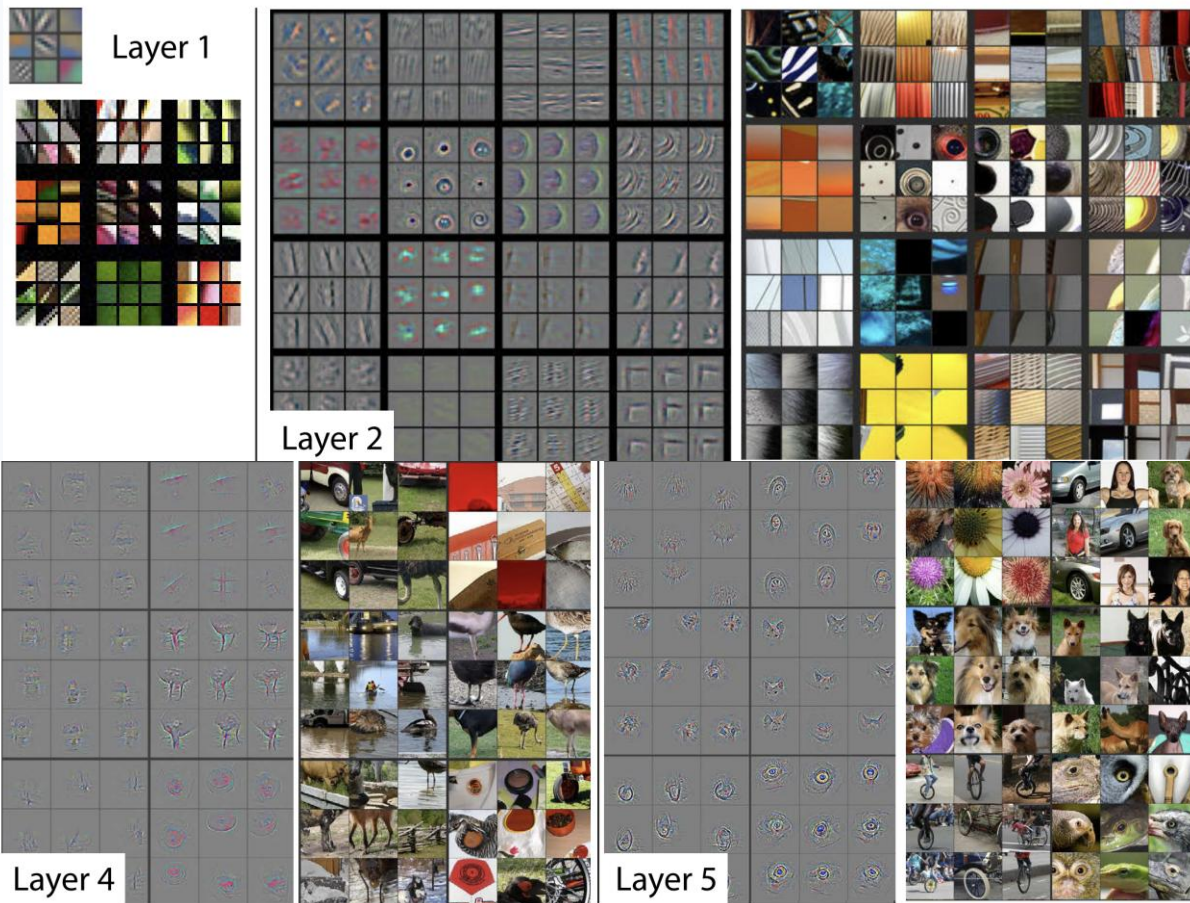
Representations inside NNs



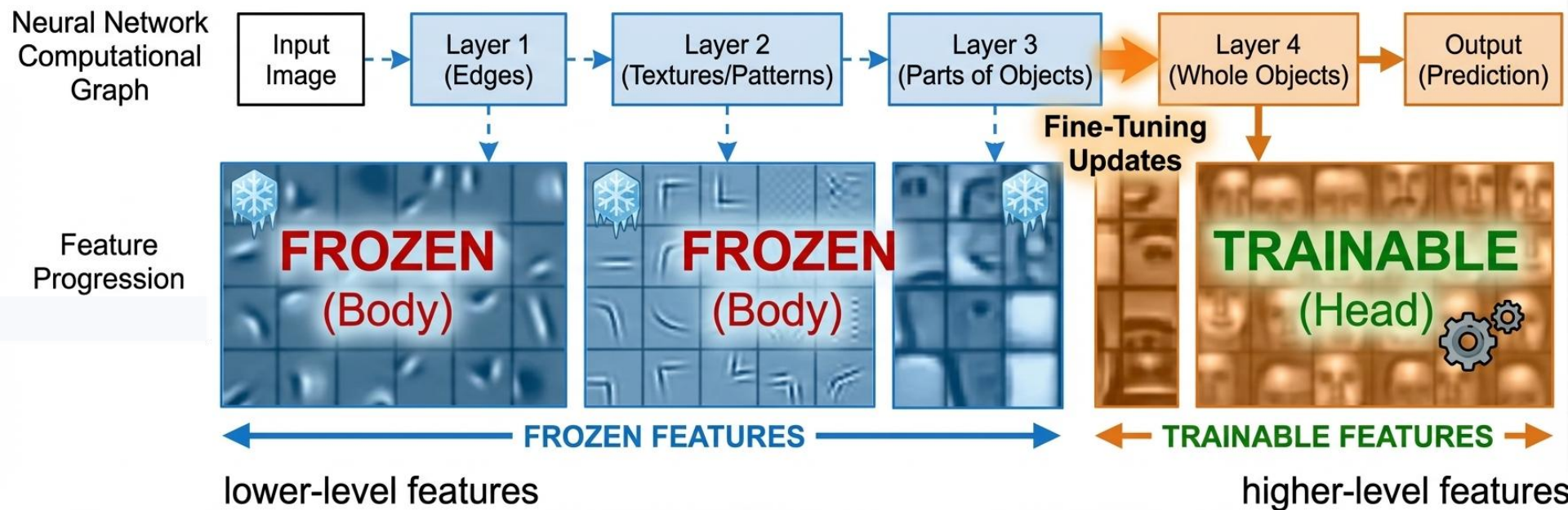
Receptive field



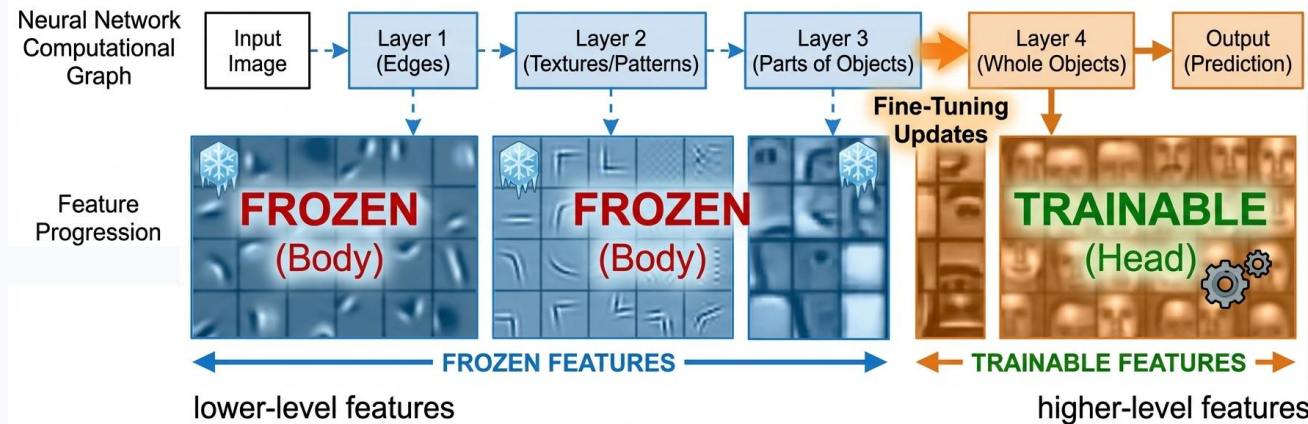
Representations inside NNs



Fine tuning



Fine tuning

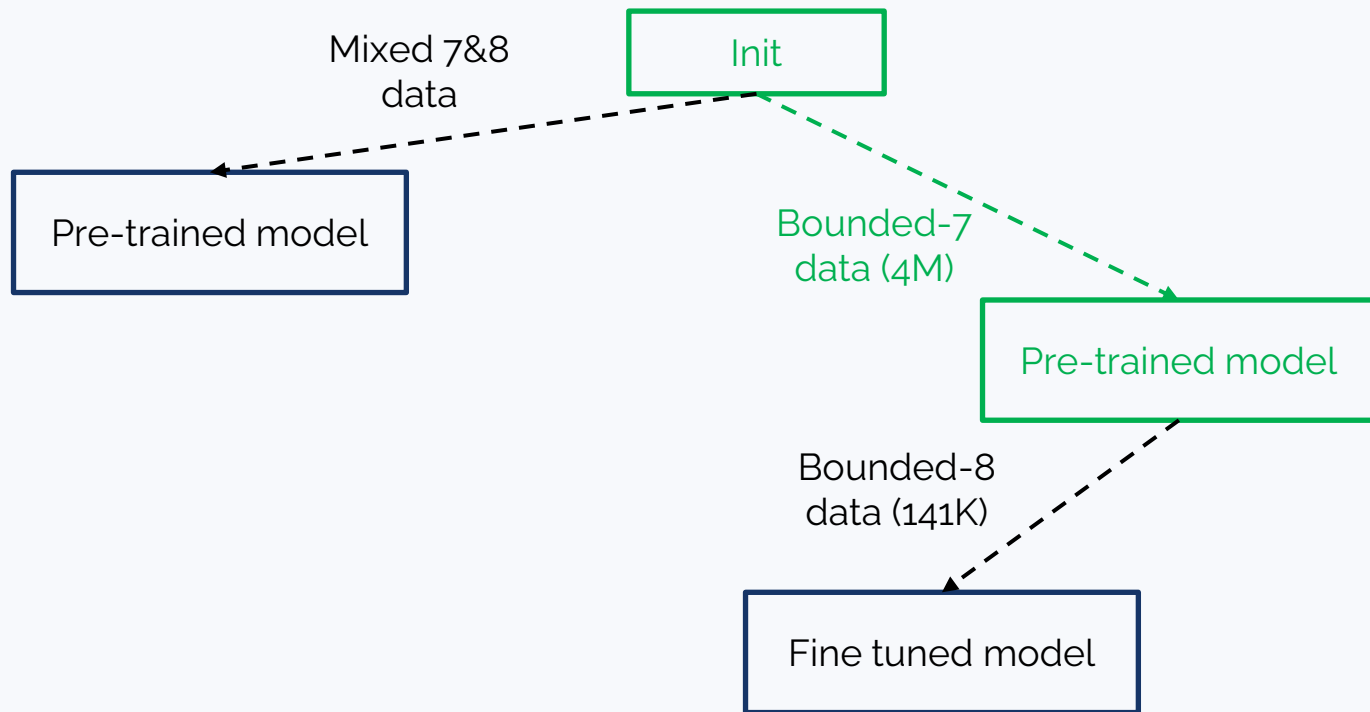


- Freeze the body, train the head
- Unfreeze the body, train both at (much) lower learning rate

Catastrophic forgetting

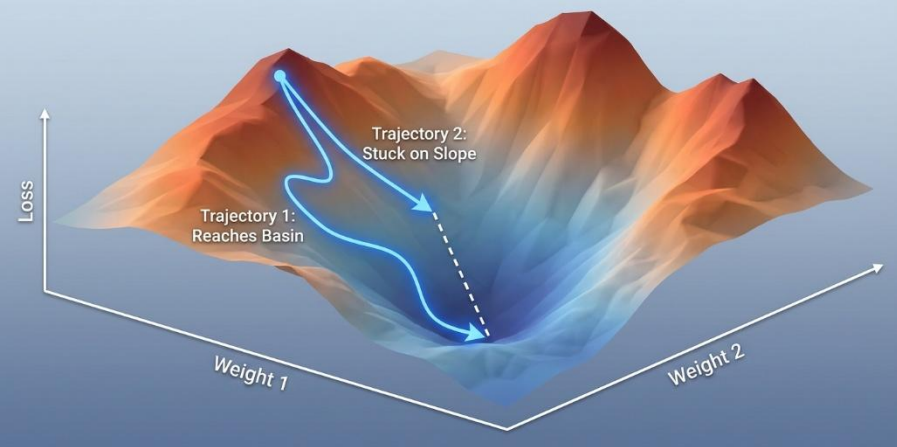
- Training on new tasks/data may lead to base task quality deterioration
- What to try: **replay**. Add pre-training data to fine tuning batches
- Fine tuning doesn't work for adding new knowledge to LLMs

A case of terminal singularities

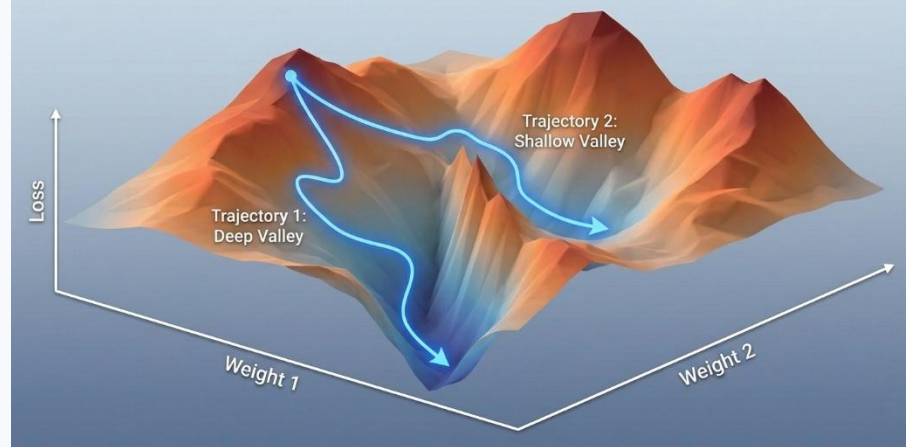


Loss barriers

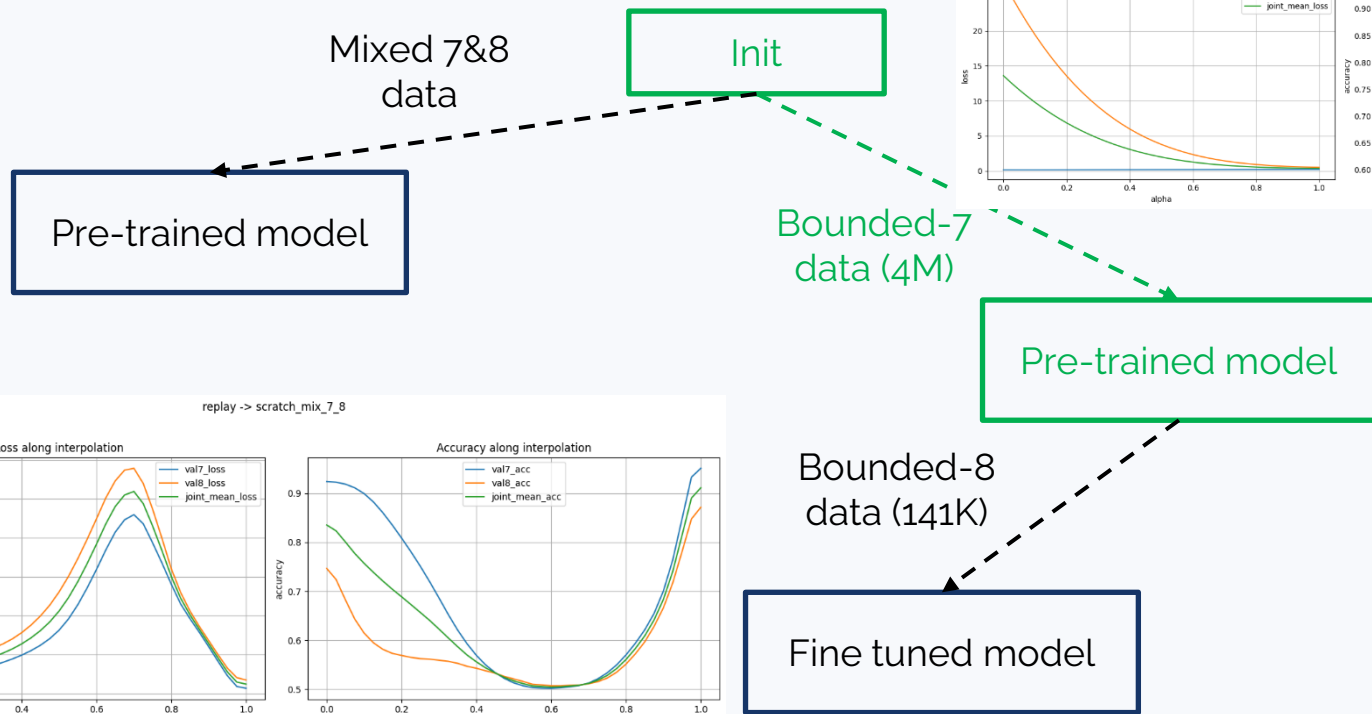
Model Training Trajectories on a Loss Landscape



Model Training Trajectories in Separate Valleys



Loss barriers



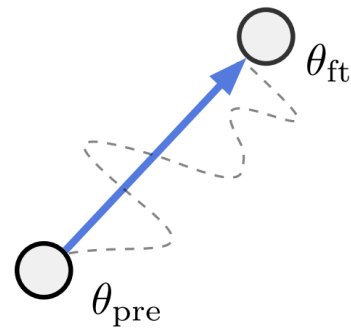
Task vectors

Task vector is a difference of weights before and after fine tuning

Forgetting via negation:

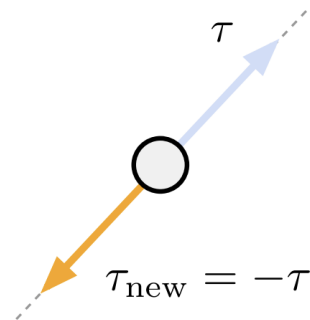
- Fine tune an LLM on toxic data
- Can now add $(-\tau)$ to other models to detoxify it without hurting base proficiency

a) Task vectors



$$\tau = \theta_{\text{ft}} - \theta_{\text{pre}}$$

b) Forgetting via negation



Example: making a language model produce less toxic content

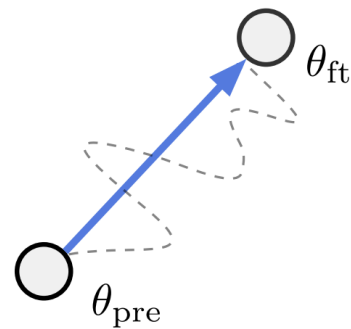
Task vectors

Forgetting via negation:

- Fine tune an LLM on toxic data
- Can now add $(-\tau)$ to other models to detoxify it without hurting base proficiency

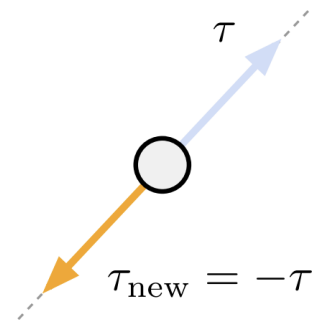
Method	% toxic generations ($\downarrow$)	Avg. toxicity score ($\downarrow$)	WikiText-103 perplexity ($\downarrow$)
Pre-trained	4.8	0.06	16.4
Fine-tuned	57	0.56	16.6
Gradient ascent	0.0	0.45	$>10^{10}$
Fine-tuned on non-toxic	1.8	0.03	17.2
Random vector	4.8	0.06	16.4
Negative task vector	0.8	0.01	16.9

a) Task vectors



$$\tau = \theta_{\text{ft}} - \theta_{\text{pre}}$$

b) Forgetting via negation



Example: making a language model produce less toxic content

Task vector arithmetic

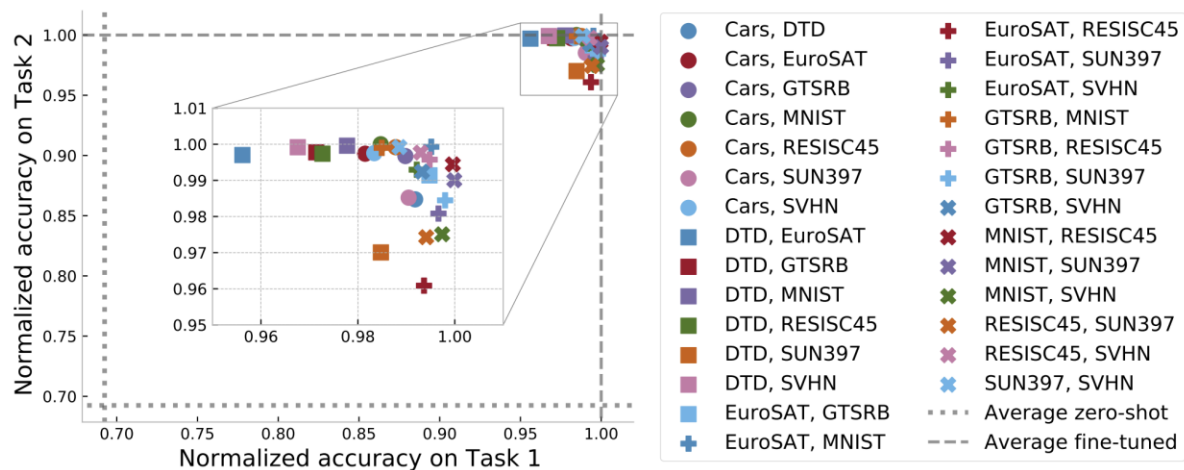
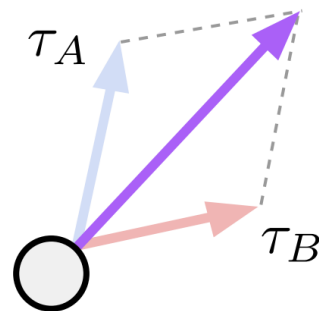


Figure 2: **Adding pairs of task vectors** from image classification tasks. Adding task vectors from two tasks improves accuracy on both, resulting in a single model that is competitive with using two specialized fine-tuned models.

c) Learning via addition

$$\tau_{\text{new}} = \tau_A + \tau_B$$



Example: building a multi-task model

Task vector arithmetic

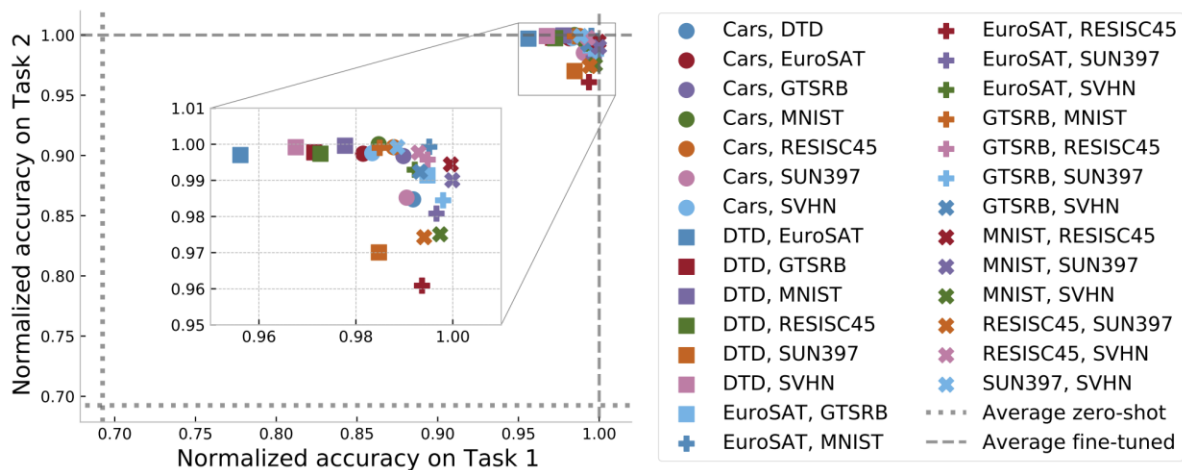
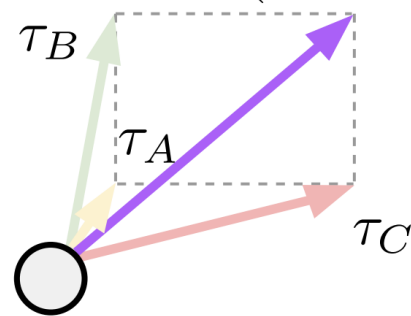


Figure 2: **Adding pairs of task vectors** from image classification tasks. Adding task vectors from two tasks improves accuracy on both, resulting in a single model that is competitive with using two specialized fine-tuned models.

d) Task analogies

$$\tau_{\text{new}} = \tau_C + (\tau_B - \tau_A)$$



Example: improving domain generalization

SERP (Spherical Interpolation)

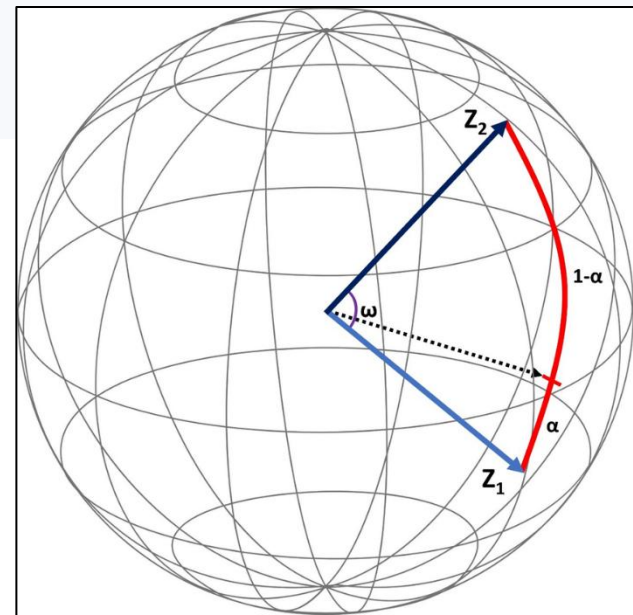
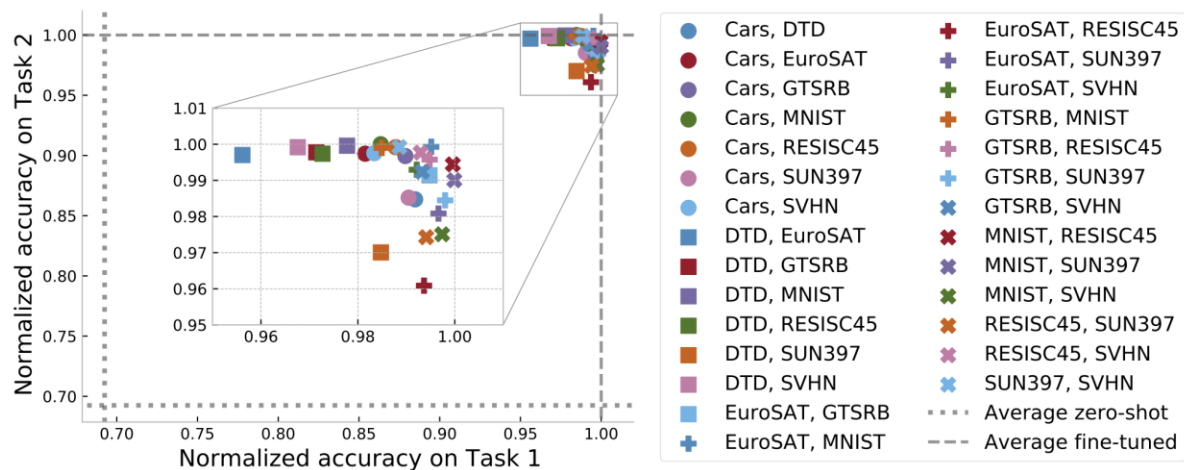


Figure 2: **Adding pairs of task vectors** from image classification tasks. Adding task vectors from two tasks improves accuracy on both, resulting in a single model that is competitive with using two specialized fine-tuned models.

Image borrowed from:

https://www.researchgate.net/figure/Illustration-of-Spherical-Linear-Interpolation-SLERP-Given-two-latent-vectors-z1-and_fig2_394292745

Physically Informed Neural Networks

Solving an ODE with an LLM

Take an ODE

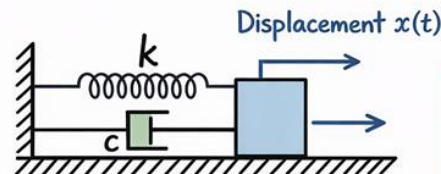
$$\ddot{x}(t) + c\dot{x}(t) + kx(t) = 0, \quad x(0) = x_0, \quad \dot{x}(0) = v_0$$

How to solve it with neural networks?

Idea: $x(t) = N_\theta(t)$ – a neural network

Damped Harmonic Oscillator Equation

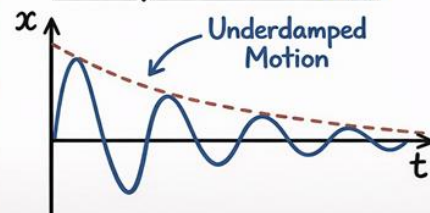
$$x''(t) + cx'(t) + kx(t) = 0$$



Physical System:
Mass-Spring-Damper
with Resistance

k = spring constant
 (stiffness of the spring)
 c = damping coefficient
 (resistance from the damper)

Example Plot of $x(t)$



C = damping coefficient
 (resistance from the damper)

Solving an ODE with an LLM

Take an ODE

$$\ddot{x}(t) + c\dot{x}(t) + kx(t) = 0, \quad x(0) = x_0, \quad \dot{x}(0) = v_0$$

How to solve it with neural networks?

Idea: $x(t) = N_\theta(t)$ – a neural network

Autograd would compute both $\ddot{x}$ and $\dot{x}$ for us!

The loss will be, for a set of **collocation points**:

$$\mathcal{L} = \frac{1}{C} \sum_{j=1}^C \left(\ddot{x}(t_j) + c\dot{x}(t_j) + kx(t_j) \right)^2$$

Solving an ODE with an LLM

Take an ODE

$$\ddot{x}(t) + c\dot{x}(t) + kx(t) = 0, \quad x(0) = x_0, \quad \dot{x}(0) = v_0$$

How to solve it with neural networks?

Idea: $x(t) = N_\theta(t)$ – a neural network

$$\mathcal{L} = \frac{1}{C} \sum_{j=1}^C (\ddot{x}(t) + c\dot{x}(t) + kx(t))^2$$

How to encode the boundary conditions?!

Solving an ODE with an LLM

Take an ODE

$$\ddot{x}(t) + c\dot{x}(t) + kx(t) = 0, \quad x(0) = x_0, \quad \dot{x}(0) = v_0$$

How to solve it with neural networks?

Idea: $x(t) = N_\theta(t)$ – a neural network

$$\mathcal{L} = \frac{1}{C} \sum_{j=1}^C (\ddot{x}(t) + c\dot{x}(t) + kx(t))^2$$

Hard constraints:

$$x(t) = x_0 + v_0 t + t^2 \cdot N_\theta(t)$$

Solving an ODE with an LLM

Take an ODE

$$\ddot{x}(t) + c\dot{x}(t) + kx(t) = 0, \quad x(0) = x_0, \quad \dot{x}(0) = v_0$$

How to solve it with neural networks?

Idea: $x(t) = N_\theta(t)$ – a neural network

$$\mathcal{L} = \frac{1}{C} \sum_{j=1}^C (\ddot{x}(t) + c\dot{x}(t) + kx(t))^2$$

Soft constraints:

$$\mathcal{L} = \mathcal{L}_{\text{ODE}} + \beta \mathcal{L}_{\text{constraints}}$$

$$\mathcal{L}_{\text{constraints}} = (N_\theta(0) - x_0)^2 + \left(\frac{d}{dt} N_\theta(0) - v_0 \right)^2$$

Problems we can solve with PINNs

- **Forward problems:** solving equations
- **Inverse problems:** given equation with unknown parameters and some experimental data, obtain both parameter values and a solution
- **Equation discovery**

Solving an inverse problem

Take an ODE

$$\ddot{x}(t) + \textcolor{red}{c}\dot{x}(t) + kx(t) = 0, \quad x(0) = x_0, \quad \dot{x}(0) = v_0$$

Unknown! 

You also have $x(t_1), \dots, x(t_N)$

Train $x(t) = N_\theta(t)$ and c :

Physics loss:

$$\mathcal{L} = \frac{1}{C} \sum_{j=1}^C (\ddot{x}(t) + c\dot{x}(t) + kx(t))^2$$

Data loss:

$$\mathcal{L}_{\text{data}} = \sum_i (N_\theta(t) - x(t_i))^2$$
$$\mathcal{L} = \mathcal{L}_{\text{ODE}} + \beta \mathcal{L}_{\text{data}}$$

Let's look at the code!

Equation discovery

Example: Physics-informed Deep Learning (PiDL)

Data: Measured data (x_i, t_i, u_i) and collocation points (z_j, s_j) .

Search an equation in the form:

$$u_t + \mathcal{F}(u, u^2, \dots, \nabla_x u, \nabla_x^2 u, \nabla_x u \cdot u, \dots; \lambda) = p$$

Rewrite it as:

$$u_t = \phi \Lambda,$$

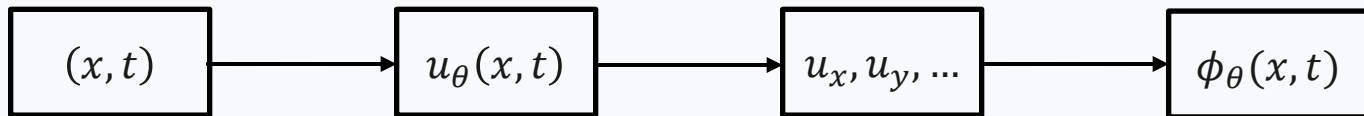
where Λ is a (hopefully sparse) numerical vector and ϕ is a *library* of functions

$$\phi = (1, u, u^2, \dots, u_x, u_y, \dots, u^3 \odot u_{xy}, \dots, \sin(u), \dots)$$

Train **both** Λ (**equation**) and u (**solution**).

Physics-informed Deep Learning (PiDL)

Data: Measured data (x_i, t_i, u_i) and collocation points (z_j, s_j) .



Losses:

$$\mathcal{L}_{\text{data}} = \frac{1}{N} \sum_i \|u_\theta(x_i, t_i) - u_i\|_2^2$$

$$\mathcal{L}_{\text{phys}} = \frac{1}{C} \sum_j \|u_t(z_j, s_j) - \phi_\theta(z_j, t_j)\Lambda\|_2^2$$

Pre-training: $\mathcal{L}_{\text{pre}} = \mathcal{L}_{\text{data}} + \alpha \mathcal{L}_{\text{phys}} + \gamma \|\Lambda\|_1$

ADO (alternating direction optimization):

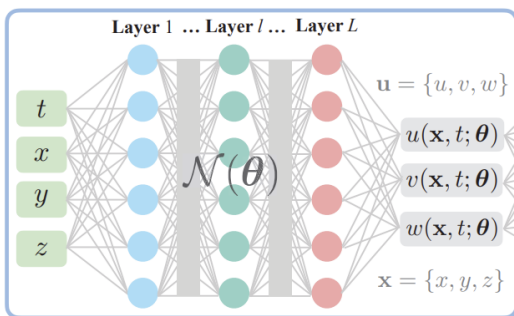
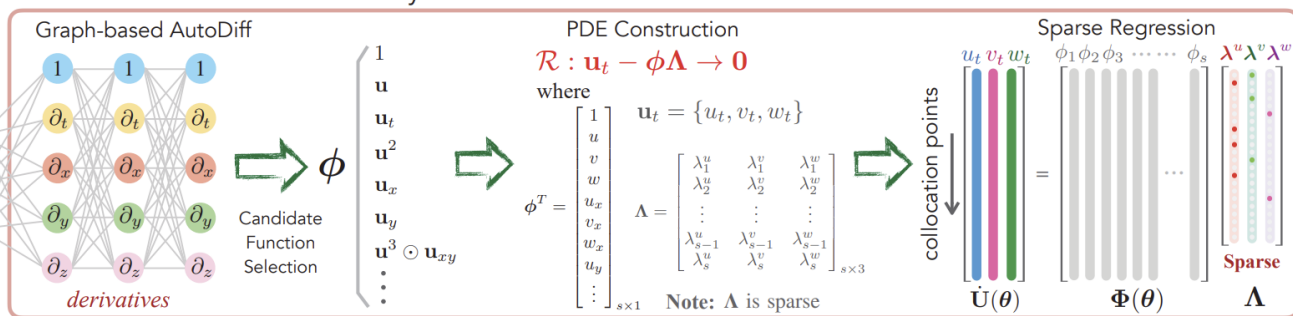
$$\mathcal{L}_{\text{phys}} + \beta \|\Lambda\|_0 \rightarrow \min_{\Lambda}$$

$$\mathcal{L}_{\text{data}} + \alpha \mathcal{L}_{\text{phys}} \rightarrow \min_{\theta}$$

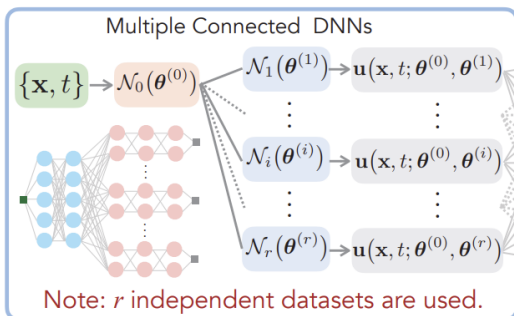
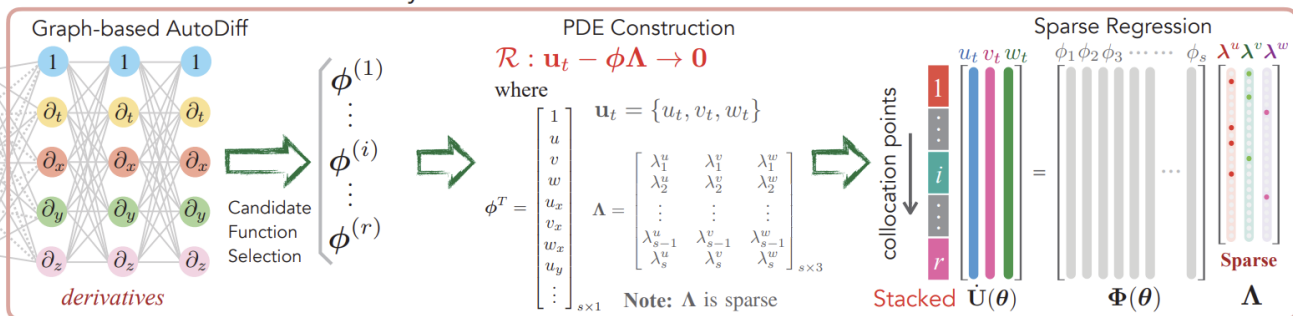
Physics-informed learning of governing equations from scarce data:

<https://arxiv.org/abs/2005.03448>

a

DNN with Unknown Parameters θ Physical Law with Unknown Parameters Λ 

b

DNNs with Unknown Parameters θ Physical Law with Unknown Parameters Λ 

c

Data Loss: $\mathcal{L}_d(\theta; \mathcal{D}_u) = \frac{1}{N_m} \|\mathbf{u}^\theta - \mathbf{u}^m\|_2^2 \rightarrow \mathcal{L}(\theta, \Lambda; \mathcal{D}_u, \mathcal{D}_c) = \underbrace{\mathcal{L}_d(\theta; \mathcal{D}_u)}_{\text{data loss}} + \underbrace{\alpha \mathcal{L}_p(\theta, \Lambda; \mathcal{D}_c)}_{\text{physics loss}} + \underbrace{\beta \|\Lambda\|_0}_{\text{regularization}} \leftarrow \text{Residual Loss: } \mathcal{L}_p(\theta, \Lambda; \mathcal{D}_c) = \frac{1}{N_c} \|\mathbf{U}(\theta) - \Phi(\theta) \Lambda\|$

measurement

total loss

collocation points

$\hat{\Lambda}_{k+1}$

Solution by **ADO**: $\hat{\Lambda}_{k+1} := \arg \min_{\Lambda} [\|\mathbf{U}(\hat{\theta}_k) - \Phi(\hat{\theta}_k) \Lambda\|_2^2 + \beta \|\Lambda\|_0]$ by STRidge $\hat{\theta}_{k+1} := \arg \min_{\theta} [\mathcal{L}_d(\theta; \mathcal{D}_u) + \alpha \mathcal{L}_p(\theta, \hat{\Lambda}_{k+1}; \mathcal{D}_c)]$ by DNN training

ADO

$\hat{\theta}_{k+1}$

Physics-informed learning of governing equations from scarce data:

<https://arxiv.org/abs/2005.03448>

Case study: cell migration after a scratch

Model: $\rho_t = \gamma \rho_{xx} + \mathcal{F}(\rho)$, where γ is the unknown diffusion coefficient and ρ is density of cells.

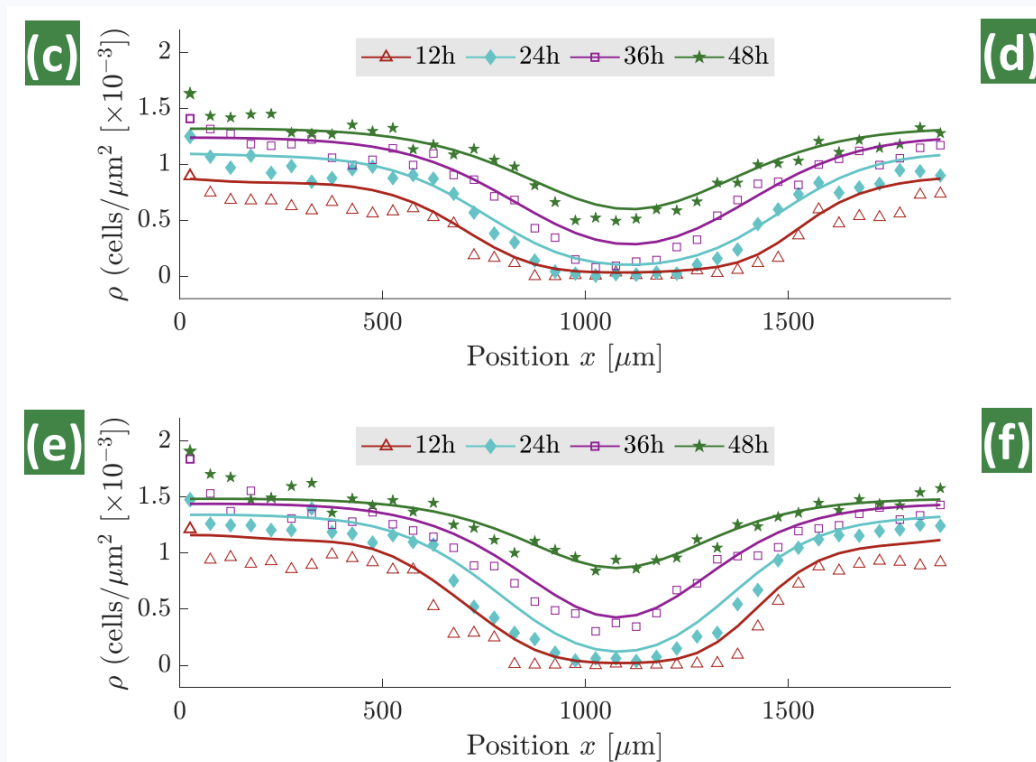
Library:

$$(1, \rho, \rho^2, \rho^3, \rho_x, \rho \rho_x, \rho^2 \rho_x, \rho^3 \rho_x)$$

Discovered:

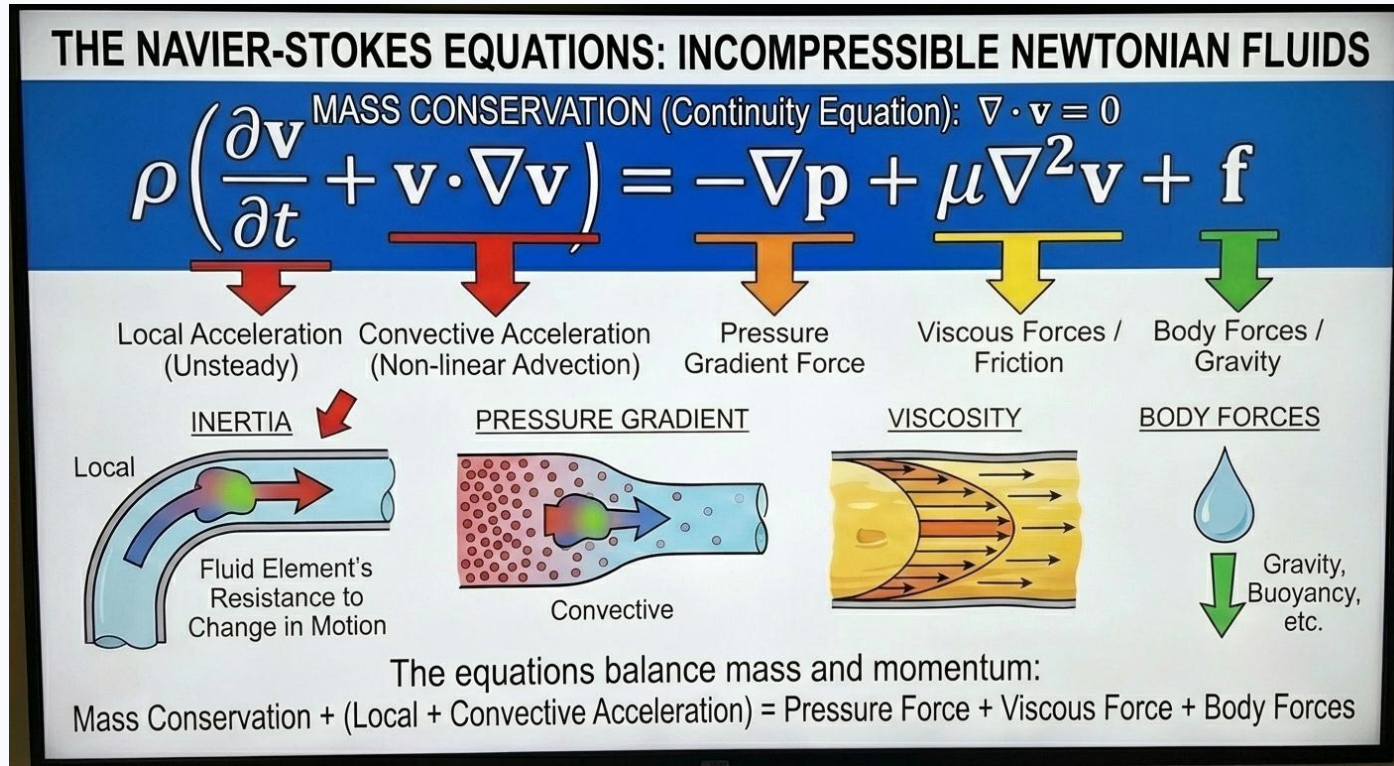
$$\rho_t = \gamma \rho_{xx} + \lambda_1 \rho + \lambda_2 \rho^2$$

Matches the Fisher-Kolmogorov model!



Discovery of unstable
singularities

Navier-Stokes equation – one of the millennium problems



2D Boussinesq equation

Models buoyancy-driven incompressible flow:

$$\nabla \cdot u = 0, \quad \theta_t + u \cdot \nabla \theta = \kappa \Delta \theta. \quad u_t + u \cdot \nabla u = -\nabla p + \nu \Delta u + \theta e_2,$$

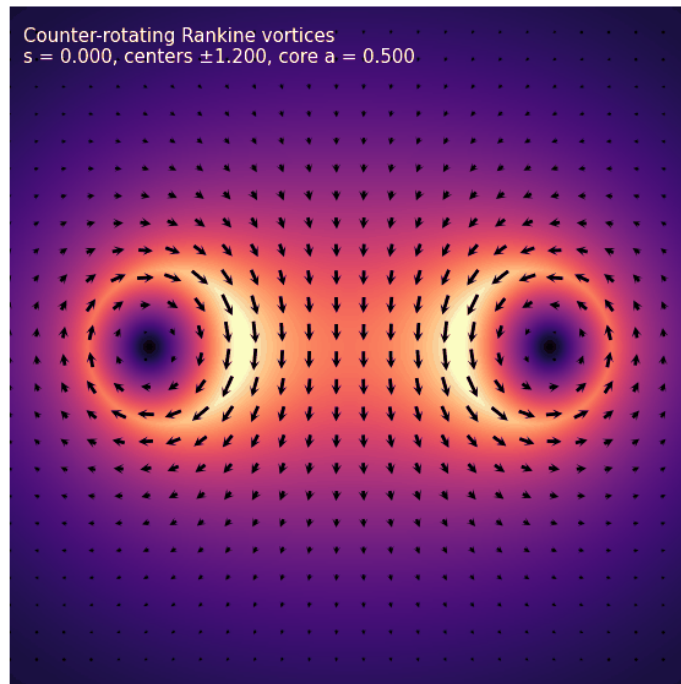
Here:

- u is the fluid velocity,
- p is pressure
- θ is typically temperature in thermal convection or density anomaly in geophysical fluids.
- θe_2 is the buoyancy force in the vertical direction.

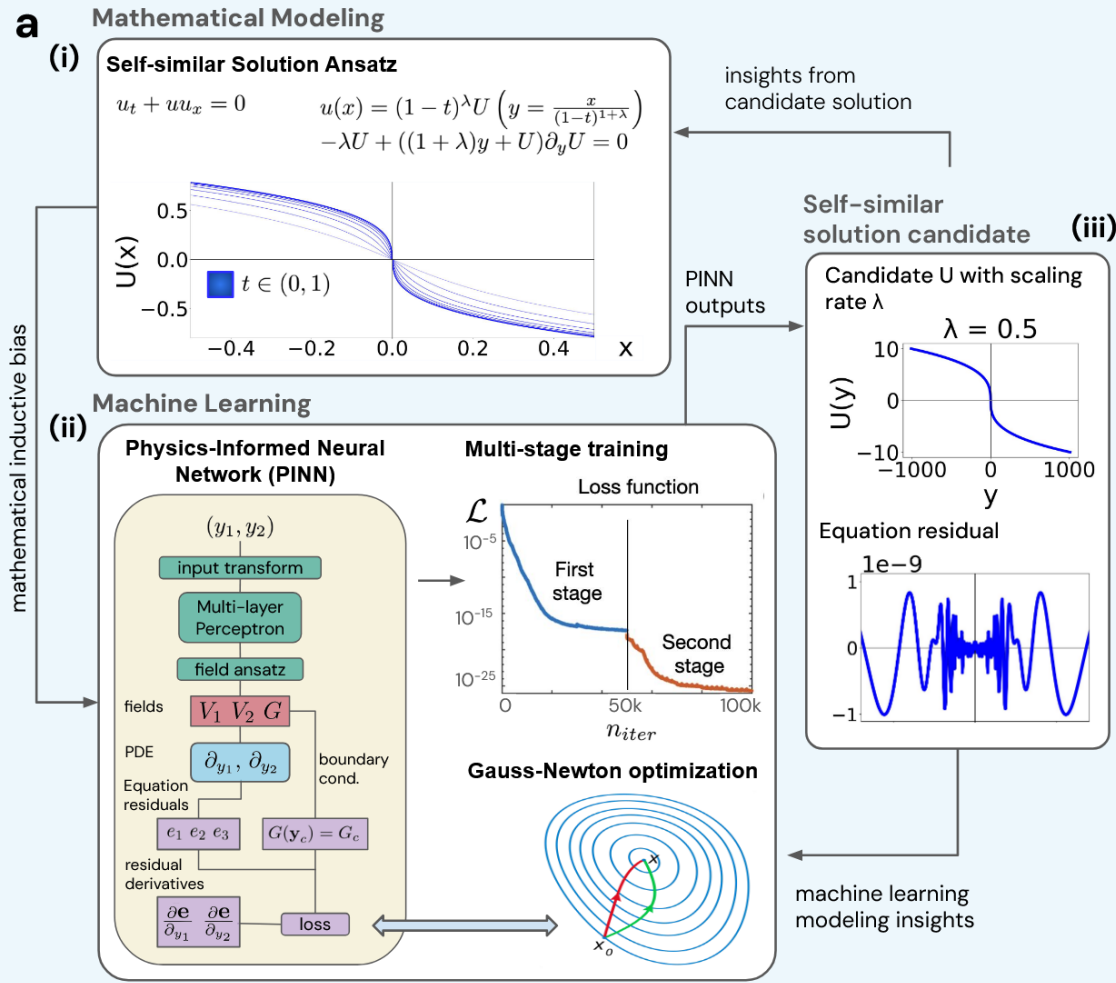
Singularities

Example: separation of a water droplet
(curvature becomes infinite)

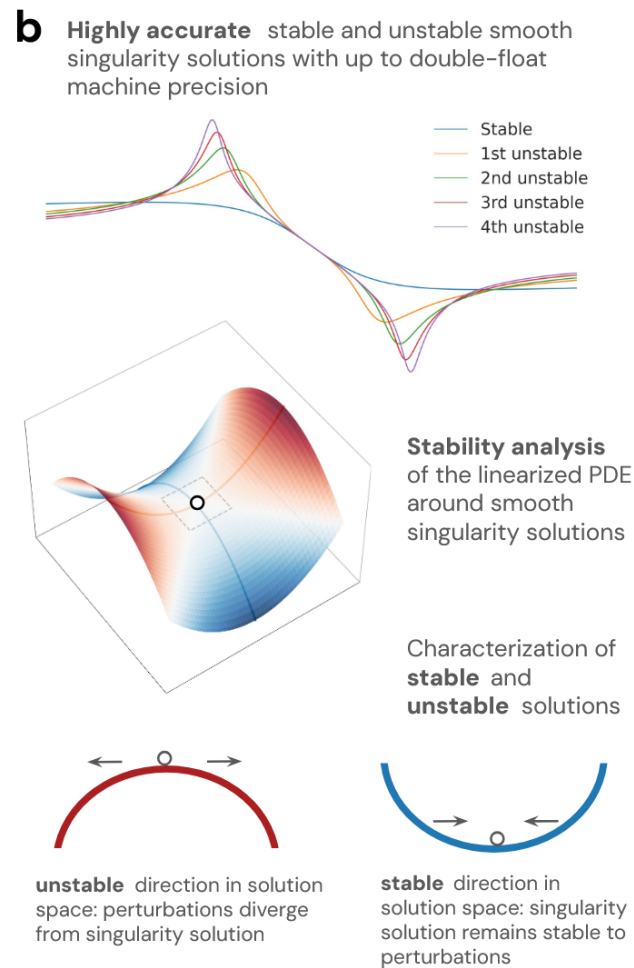
- **Stable:** remains singular under small change of initial conditions
- **Unstable:** is a very lucky coincidence



High Precision Self-Similar Solution Discovery



Solution Analysis

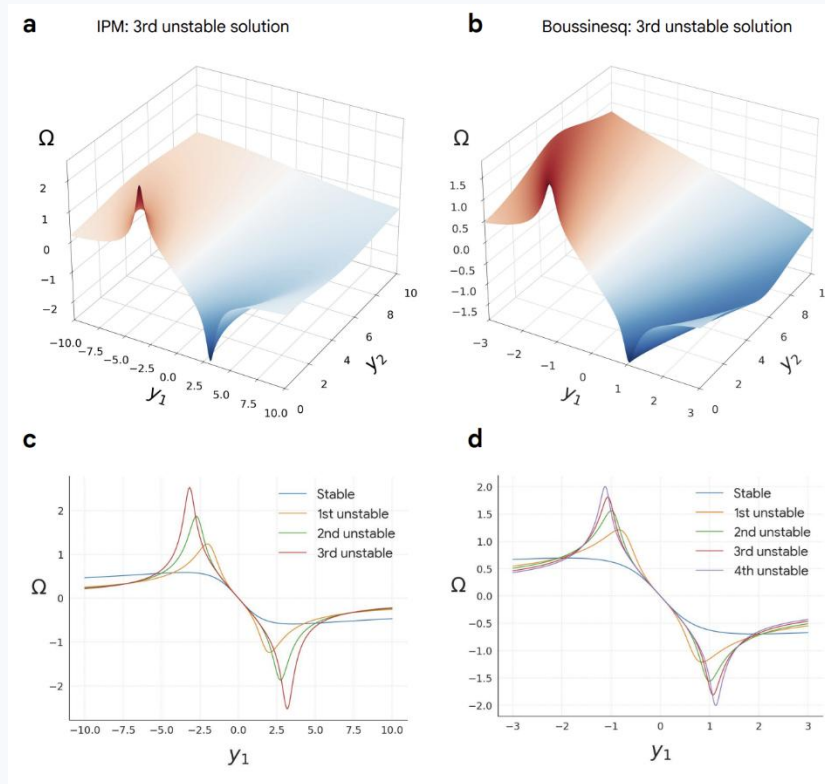


Self-similar ansatz

Model with a special function $\Phi_\theta(y)$; describe evolution in time by simple contraction, with singularity occurring at $t = 1$ and $x = 0$:

$$\phi(x, t) = (1 - t)^{k(\lambda)} \Phi_\theta \left(\frac{x}{(1 - t)^{1+\lambda}} \right)$$

The 3d plot shows *vorticity* in spatial coordinates. The two peaks correspond to two counter-rotating vortices.



Even more hard constraints for vorticity:

$$\phi(x, t) = (1 - t)^{k(\lambda)} \Phi_{\theta} \left(\frac{x}{(1 - t)^{1+\lambda}} \right)$$

New coordinates:

$$q = (1 + |y|^2)^{-1/(2(1+\lambda))}, \quad \beta = \frac{y_2}{\sqrt{1 + y_1^2 + y_2^2}}$$

Encoding more properties:

$$\Omega(y_1, y_2) = \underbrace{\left(\frac{y_1}{\sqrt{1 + y_1^2 + y_2^2}} \right)}_{\text{Odd Symmetry}} \cdot \text{NN}_{\Omega}[q, \beta] \cdot \underbrace{q(y_1, y_2)}_{\text{Asymptotic Decay}}$$

Loss functions

Pick a point y_{norm} in a
high-gradient region;
This helps to avoid trivial
zero solution

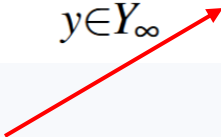


$$\mathcal{L}_{\text{d0}}(Y_0) = \frac{1}{N_{\text{d0}}N_E} \sum_{(y,\rho) \in Y_0} \rho \sum_{k=1}^{N_E} [\mathcal{R}_k(\hat{\Phi}_\theta(y), \lambda)]^2.$$

$$\mathcal{L}_{\text{d1}}(Y_1) = \frac{1}{N_{\text{d1}}N_E d} \sum_{(y,\rho) \in Y_1} \rho \sum_{k=1}^{N_E} \sum_{i=1}^d [\partial_{y_i} \mathcal{R}_k(\hat{\Phi}_\theta(y), \lambda)]^2,$$

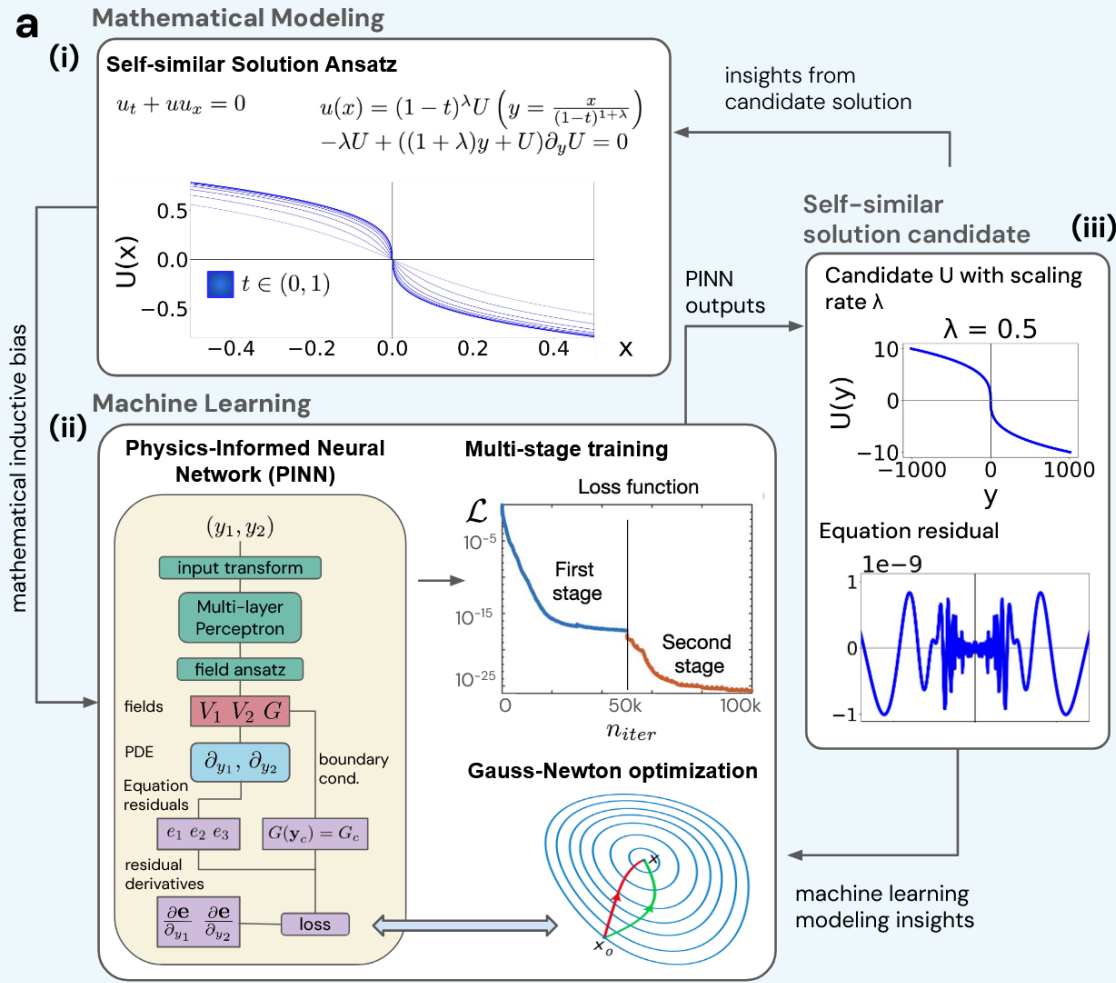
$$\mathcal{L}_{\text{d2}}(Y_2) = \frac{1}{N_{\text{d2}}N_E d^2} \sum_{(y,\rho) \in Y_2} \rho \sum_{k=1}^{N_E} \sum_{i,j=1}^d [\partial_{y_i} \partial_{y_j} \mathcal{R}_k(\hat{\Phi}_\theta(y), \lambda)]^2,$$

$$\mathcal{L}_{\text{equation}} = c_{\text{d0}} \cdot \mathcal{L}_{\text{d0}} + c_{\text{d1}} \cdot \mathcal{L}_{\text{d1}} + c_{\text{d2}} \cdot \mathcal{L}_{\text{d2}}.$$

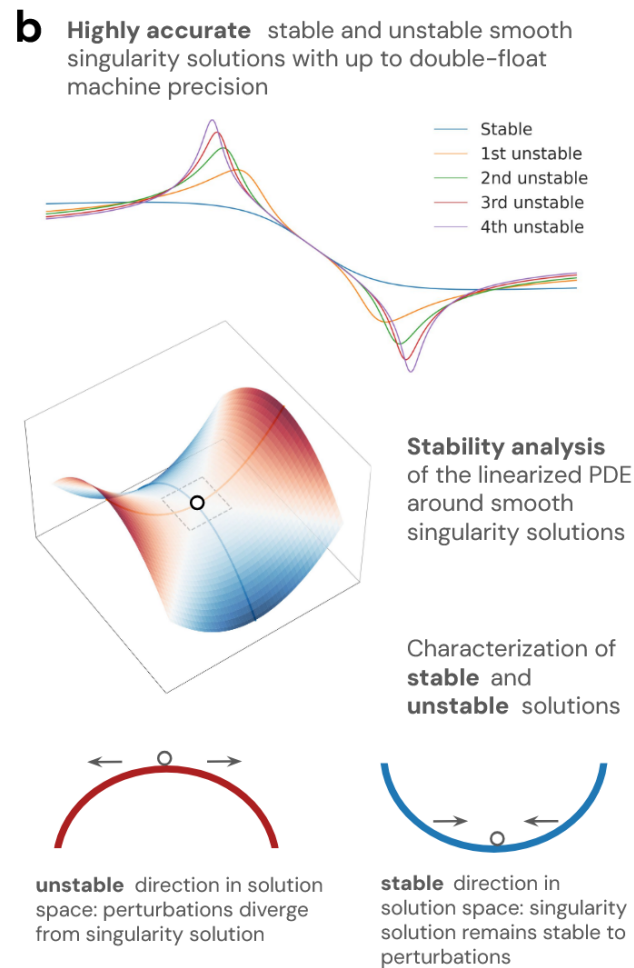
$$\mathcal{L}_{\text{data}} = \hat{c}_{\text{norm}} (\Phi_{A_0}(y_{\text{norm}}) - C_{\text{norm}})^2 + \hat{c}_{\infty} \sum_{y \in Y_{\infty}} \Phi_{A_0}(y)^2.$$


The function decays at the infinity

High Precision Self-Similar Solution Discovery



Solution Analysis



Dimensionality reduction

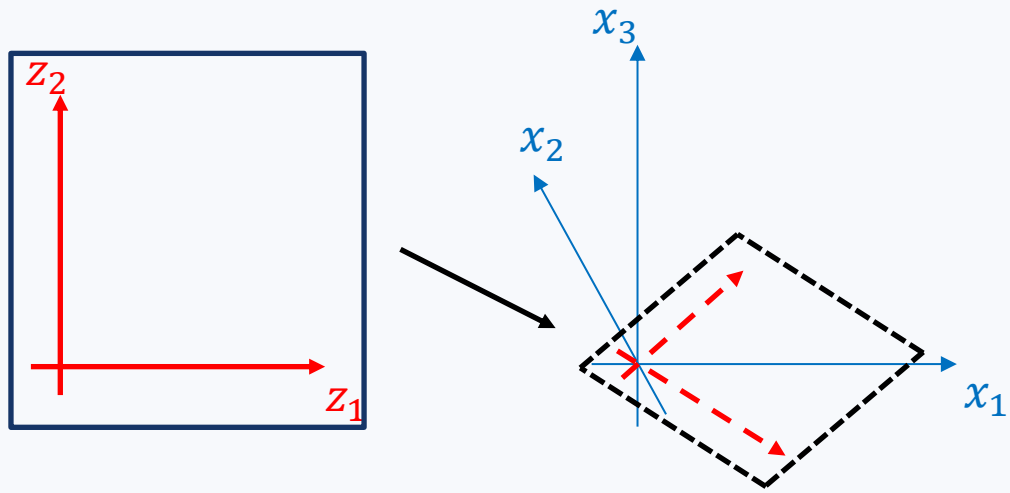
Principal component analysis

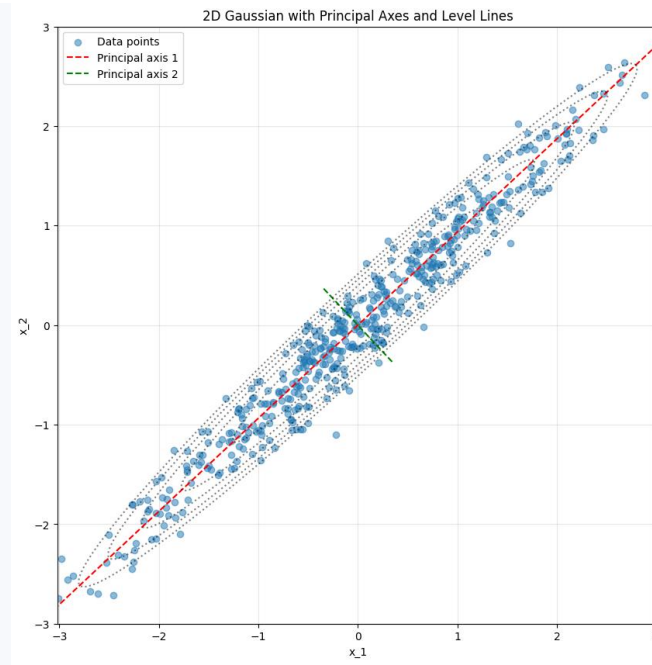
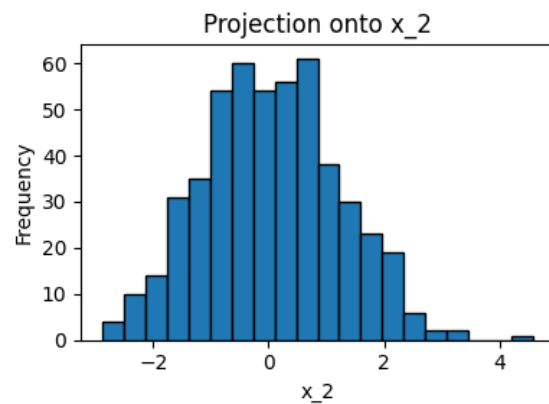
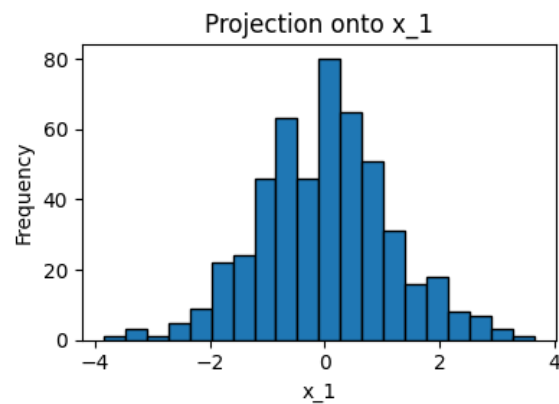
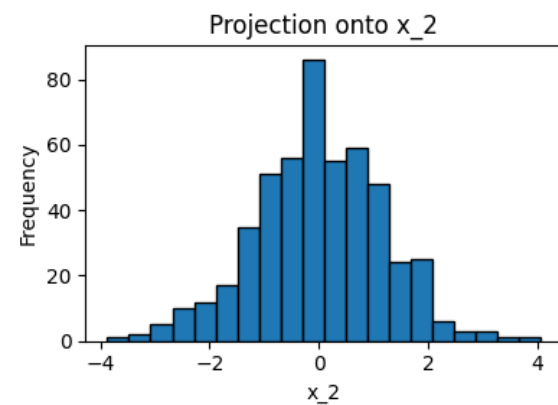
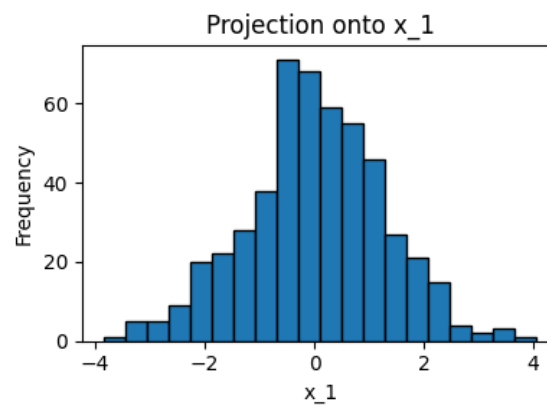
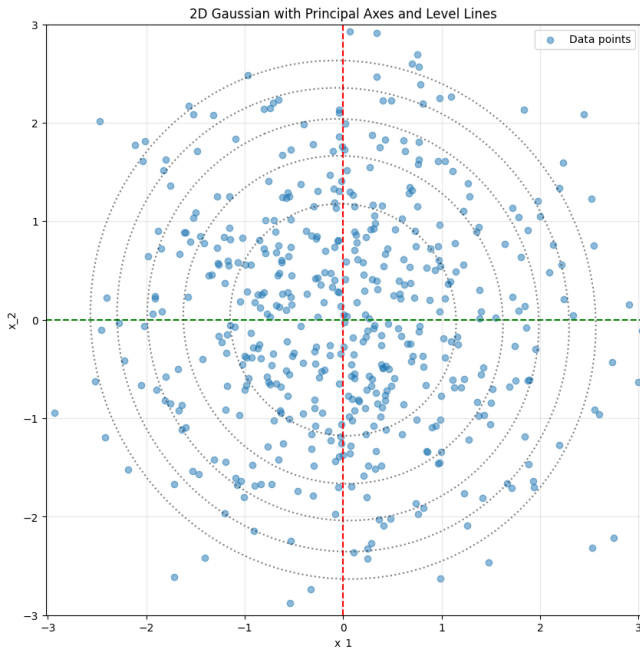
Establish

$$X^{N \times D} = Z^{N \times d} \cdot V^T$$

where z_i are new (**latent**) features of the i -th object and $d < D$.

- $Z \rightarrow Z \cdot V^T$ is feature mixing
- $Z \rightarrow U \cdot Z$ is object mixing



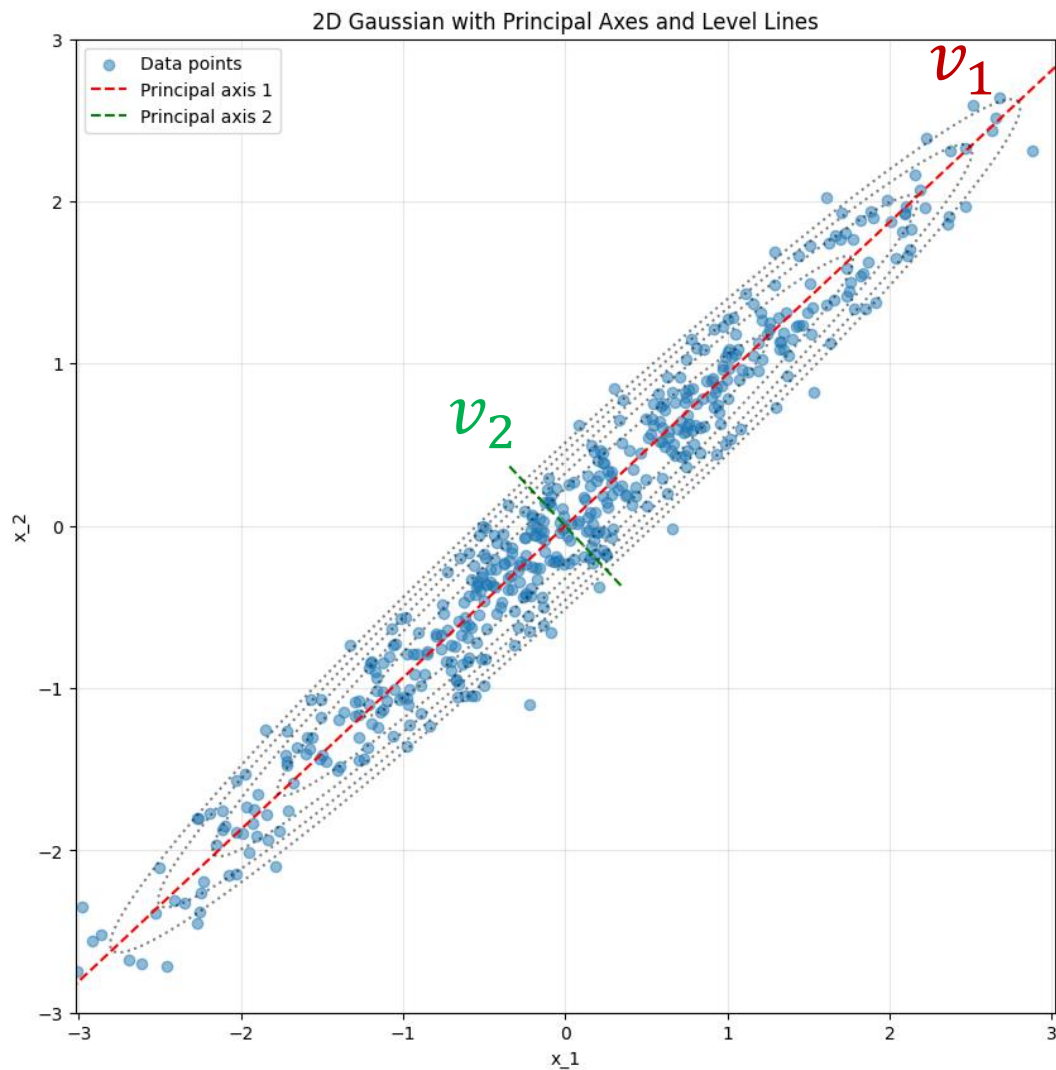


Principal component analysis

If $V = (v_1, \dots, v_D)$, then

$$V^T X^T X V = \Sigma = \text{diag}(\sigma_1, \sigma_2, \dots)$$

where $\sigma_1 \geq \sigma_2 \geq \dots$



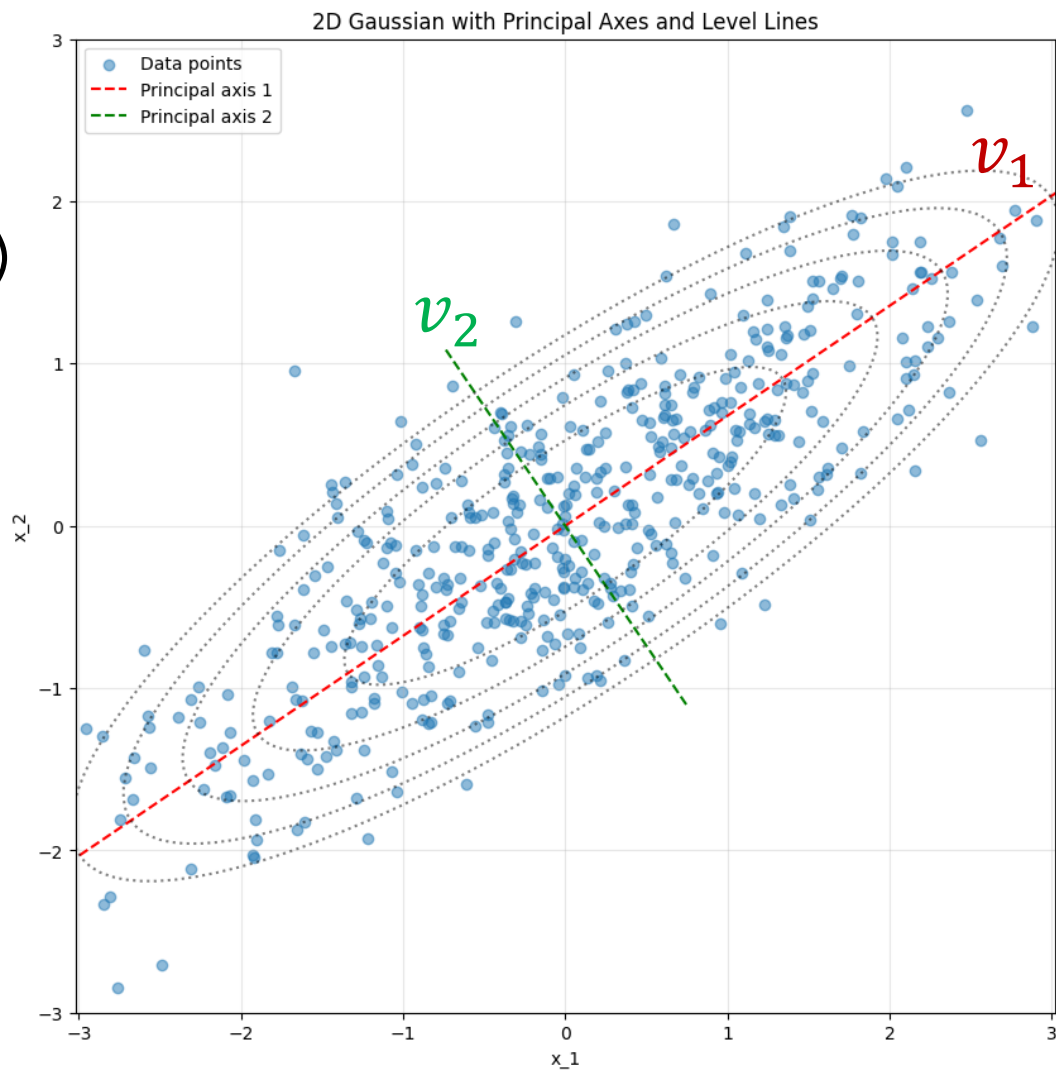
Singular value decomposition (SVD)

One may prove that

$$X = U\Sigma V^T$$

where

- U and V have orthogonal columns,
 - $\Sigma = \text{diag}(\sigma_1, \sigma_2, \dots)$
- where $\sigma_1 \geq \sigma_2 \geq \dots$



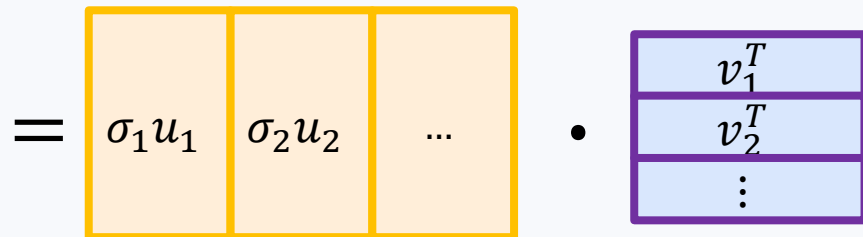
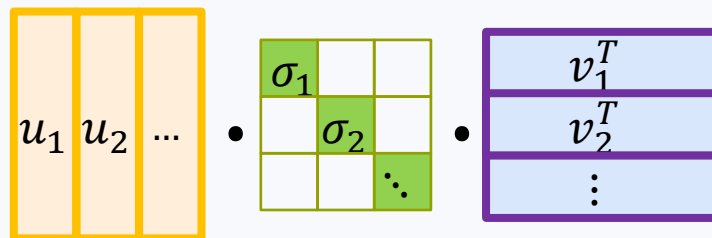
Principal component analysis

$$X = U\Sigma V^T$$

Here, V^T mixes features. We may take new feature description:

$$X \rightarrow U\Sigma$$

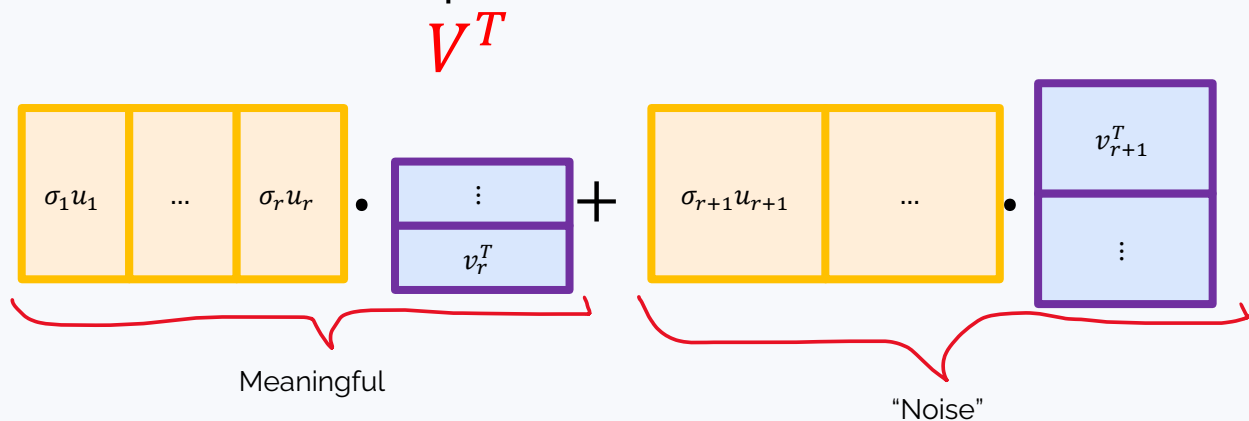
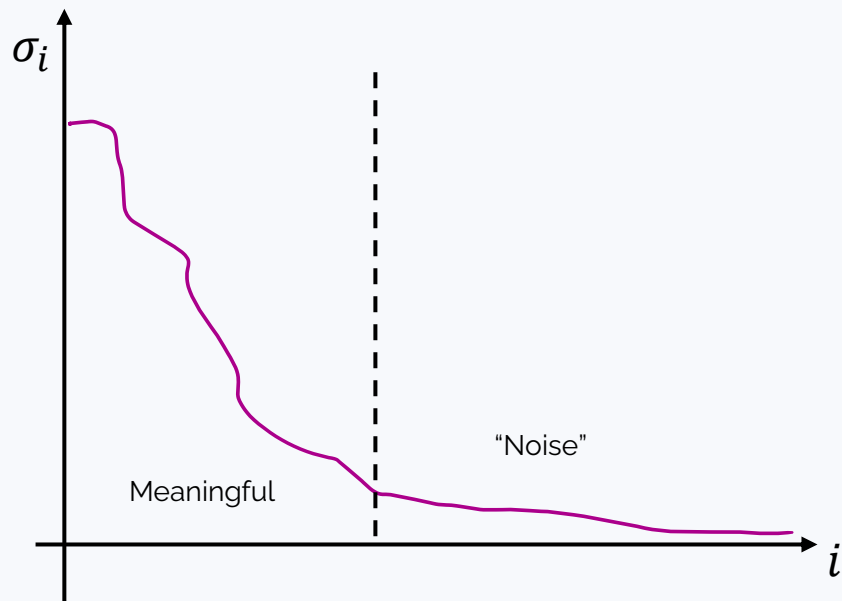
i -th object will be described by $(\sigma_i u_{i1}, \sigma_i u_{i2}, \dots)$



Principal component analysis

$$X = U\Sigma V^T$$

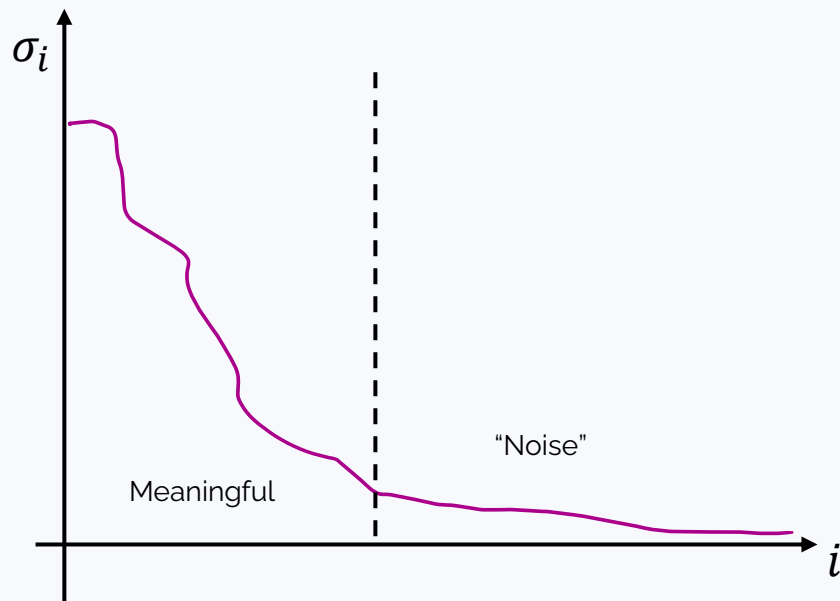
$$X \rightarrow U\Sigma$$



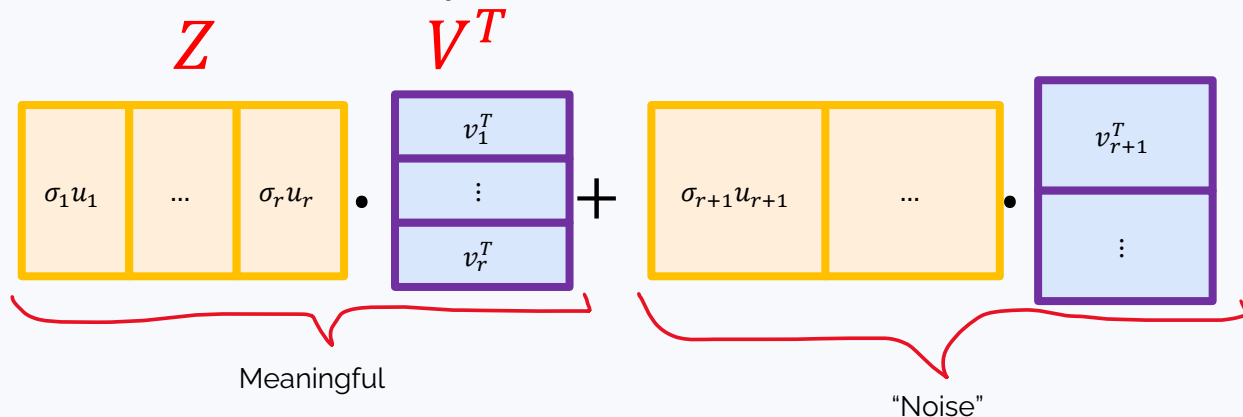
Principal component analysis

$$X = U\Sigma V^T$$

$$X \rightarrow U\Sigma$$



- $X \approx ZV^T$
- Z – new object-feature matrix
- V^T – mixing matrix



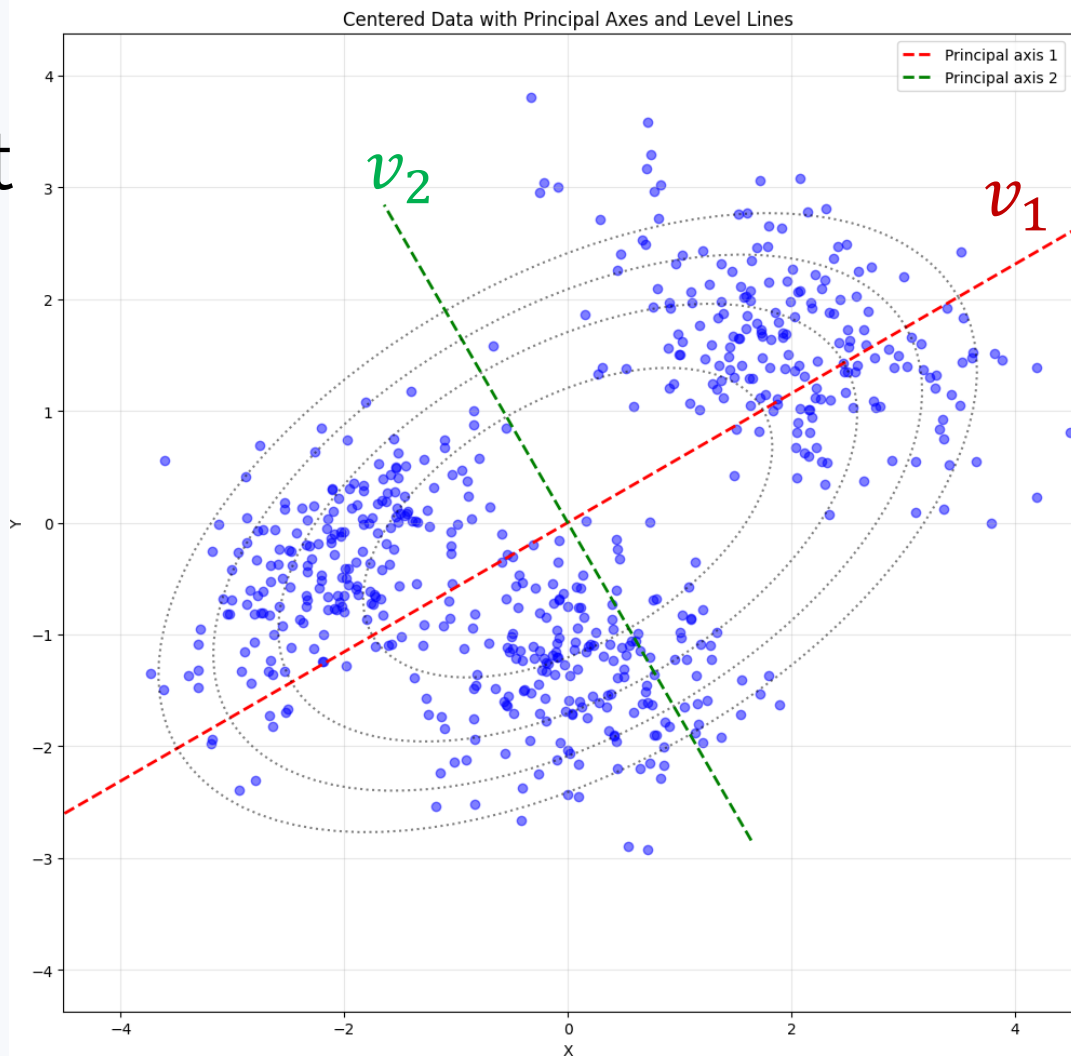
Principal component analysis

$$X = U\Sigma V^T$$

Here, V^T mixes features. We may take new feature description:

$$X \rightarrow U\Sigma$$

i -th object will be described by $(\sigma_i u_{i1}, \sigma_i u_{i2}, \dots)$

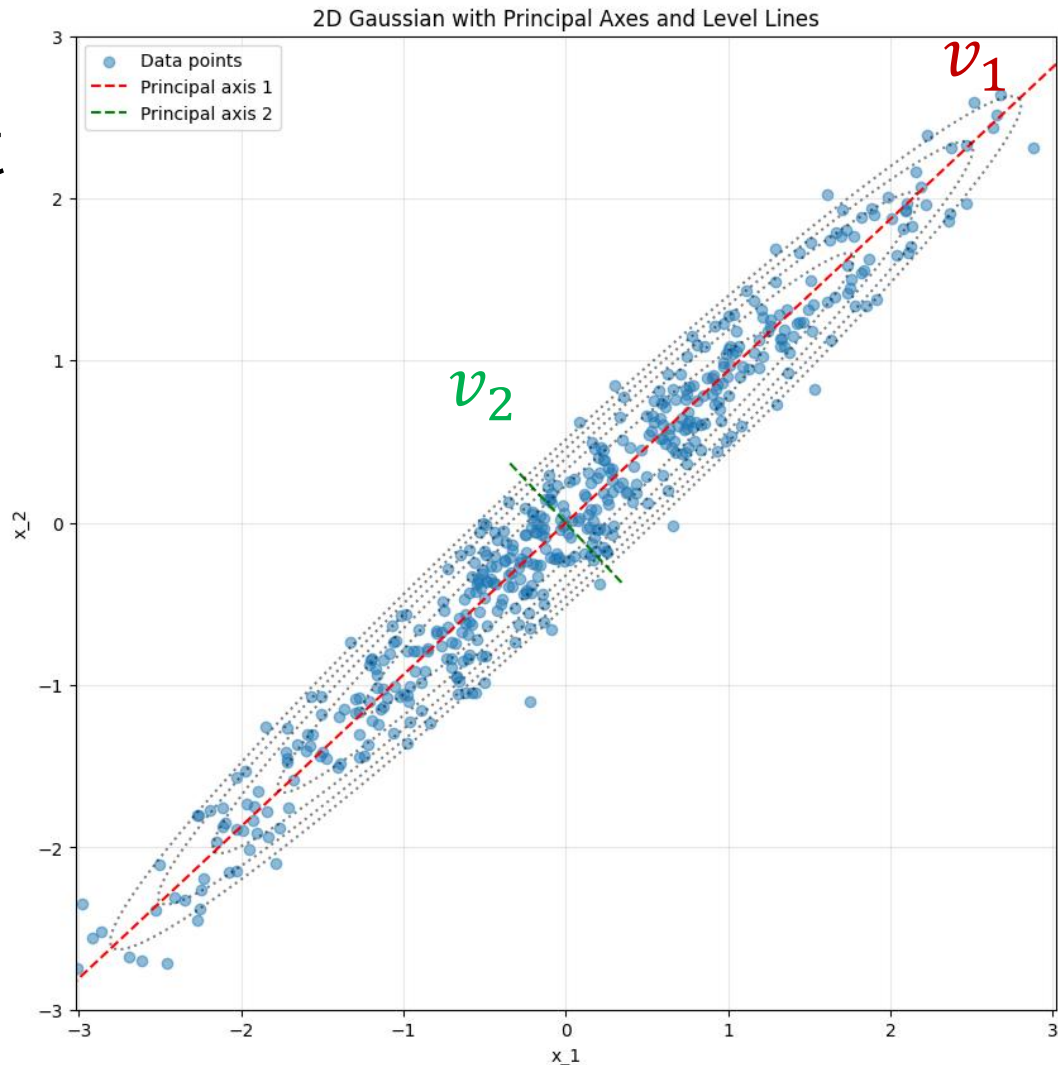


Principal component analysis

$$X \rightarrow U\Sigma$$

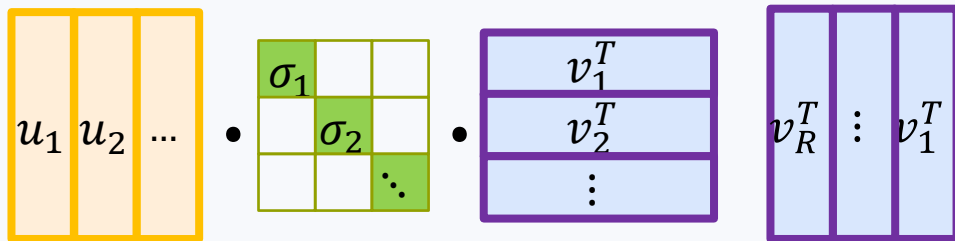
i -th object will be described by $(\sigma_i u_{i1}, \sigma_i u_{i2}, \dots, \sigma_i u_{id})$

In some sense, this is the best d -dimensional approximation.



Mapping to latent feature space and back

$$X = U\Sigma V^T \rightarrow U\Sigma \boxed{V^T \cdot V} \stackrel{=I}{=} U\Sigma$$

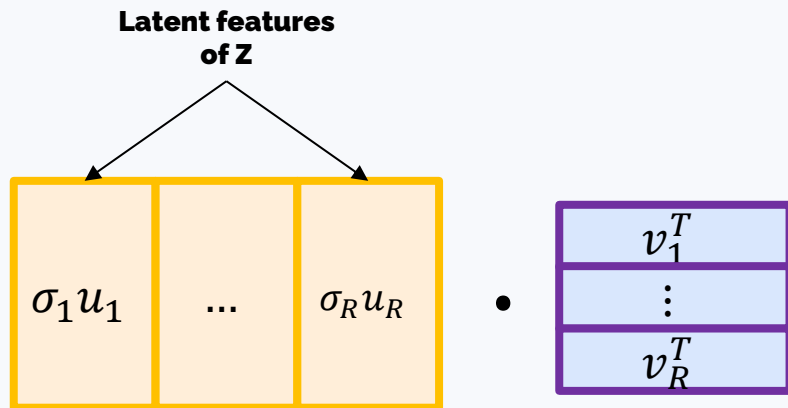


If we want to get R latent features:

$$U\Sigma[:, :R] = U\Sigma V^T \cdot V[:, :R]$$

$$X_{test} \rightarrow X_{test} \cdot V[:, :R]$$

$$\text{Back: } Z \rightarrow Z \cdot V^T[:, R, :]$$



Pros and cons of PCA

Pros:

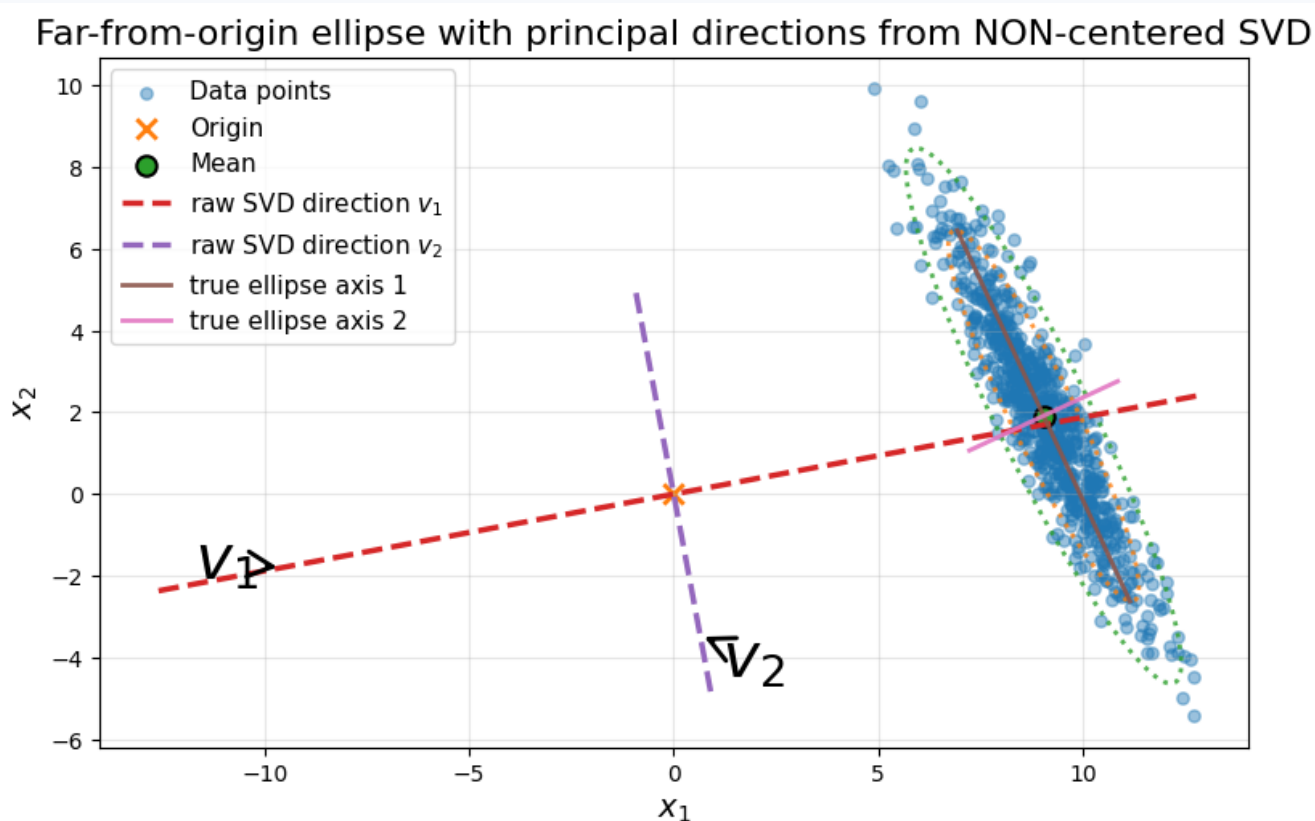
- Simple and effective
- Still encountered in papers

Cons:

- May not work well with non-ellipsoid (non-gaussian data)

Don't forget to normalize the data!

SVD for non-centralized data



tSNE

Map distances to probabilities:

Original data	Low-dimensional data
$p_{j i} = \frac{\exp(-\ x_i - x_j\ ^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\ x_i - x_k\ ^2 / 2\sigma_i^2)}$ $p_{ij} = \frac{1}{2N} (p_{j i} + p_{i j}); \quad p_{ii} = 0$	$q_{ij} = \frac{(1 + \ x_i - x_j\ ^2)^{-1}}{\sum_{k \neq l} (1 + \ x_l - x_k\ ^2)^{-1}}$ Heavier tails $\Rightarrow$ less probability of points cluttering together

Minimize:

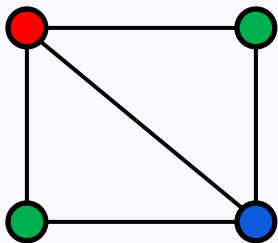
$$\text{KL}(P \parallel Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

An example: visualizing
chromatic polynomials
of graphs

Proper colorings and chromatic polynomials

Proper coloring:
Adjacent vertices
colored differently

Number of colorings in q colors
is a polynomial in q
It's called **chromatic polynomial** of
the graph



$$P_G(q) = q(q - 1)(q - 2)^2$$

Let's look at the code!

The plan for the rest of the course

April 8	Autoencoders & RNNs
April 15	Transformers
April 22	LLMs & Reasoning
April 29	Agents & MCTS
May 6	RL-1
May 13	RL-2
May 20	Algorithmic optimization & AlphaEvolve
May 27	RL-3 & Wrap Up